Toward truly-autonomous distributed computing environment

Hidehiro Kanemitsu, Ph. D,

Global Education Center, Waseda University, Japan.

kanemih@waseda.jp

Self Intro.

- **Name**: Hidehiro Kanemitsu, Ph. D,
- **Affiliation**: Associate Professor, Global Education Center, Waseda University, Japan.
  - Past: Tokyo University of Technology, Japan

- **Research Topic**: Parallel and Distributed Computing.
  - **Task Scheduling** (Allocation and ordering tasks to computational resources)
  - **ICN** (Information-Centric Networking)
  - **NFV**(Network Function Virtualization)
  - **P2P**(Efficient content lookup using DHT)
  - **MapReduce** (Resource provisioning)

Table of contents

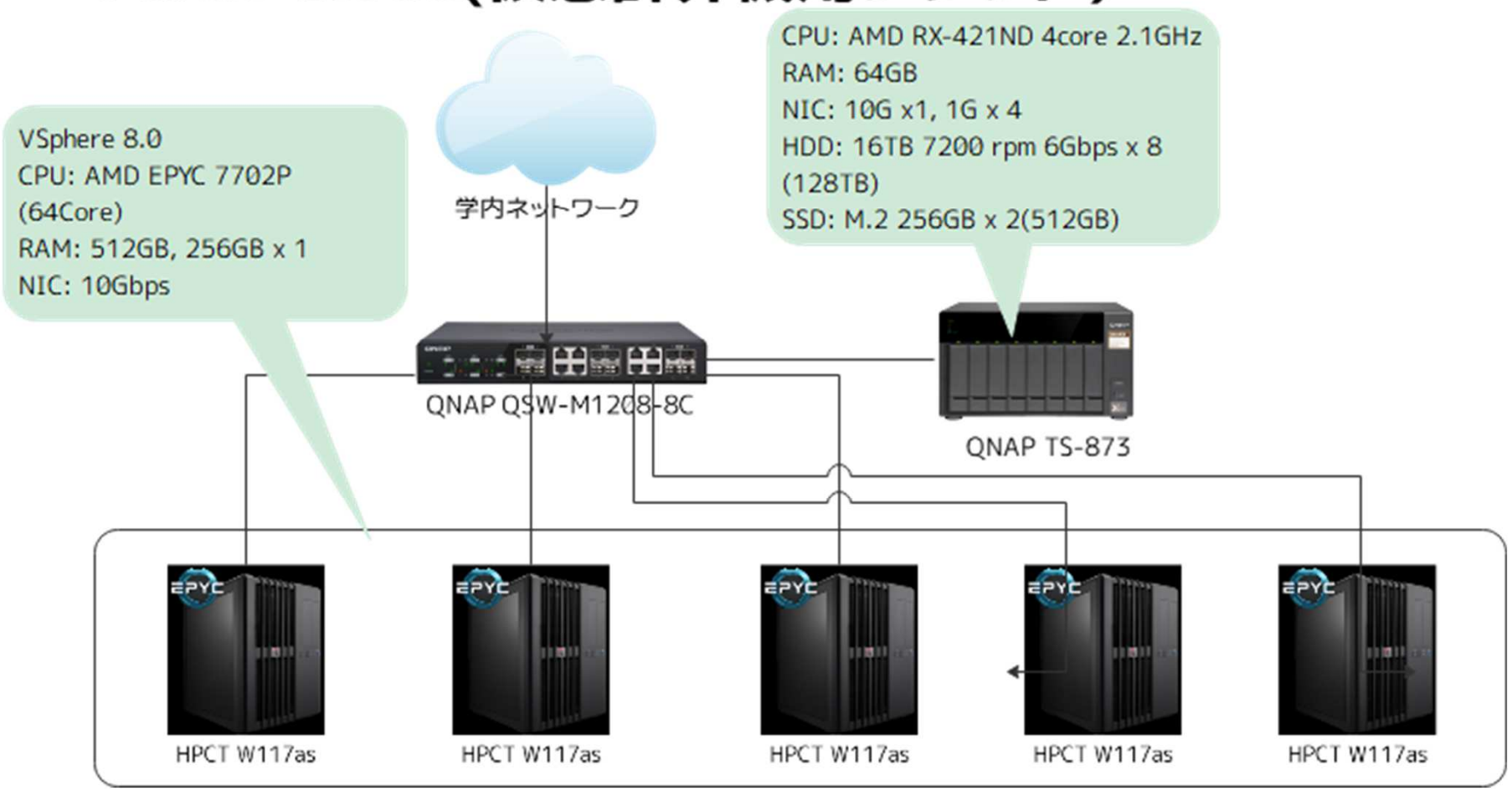
- Cloud computing and its application to AI infrastructure.

- Introduction to national projects
  - Cloud/virtualization for smart cities
  - Co-creating digital twin
  - Performance improvement for P2P
  - Web3-enabled digital twin/distributed DB

- Those involved technologies
  - Task scheduling
  - DHT(Distributed Hash Table) for P2P
  - ICN(Information-Centric Networking)

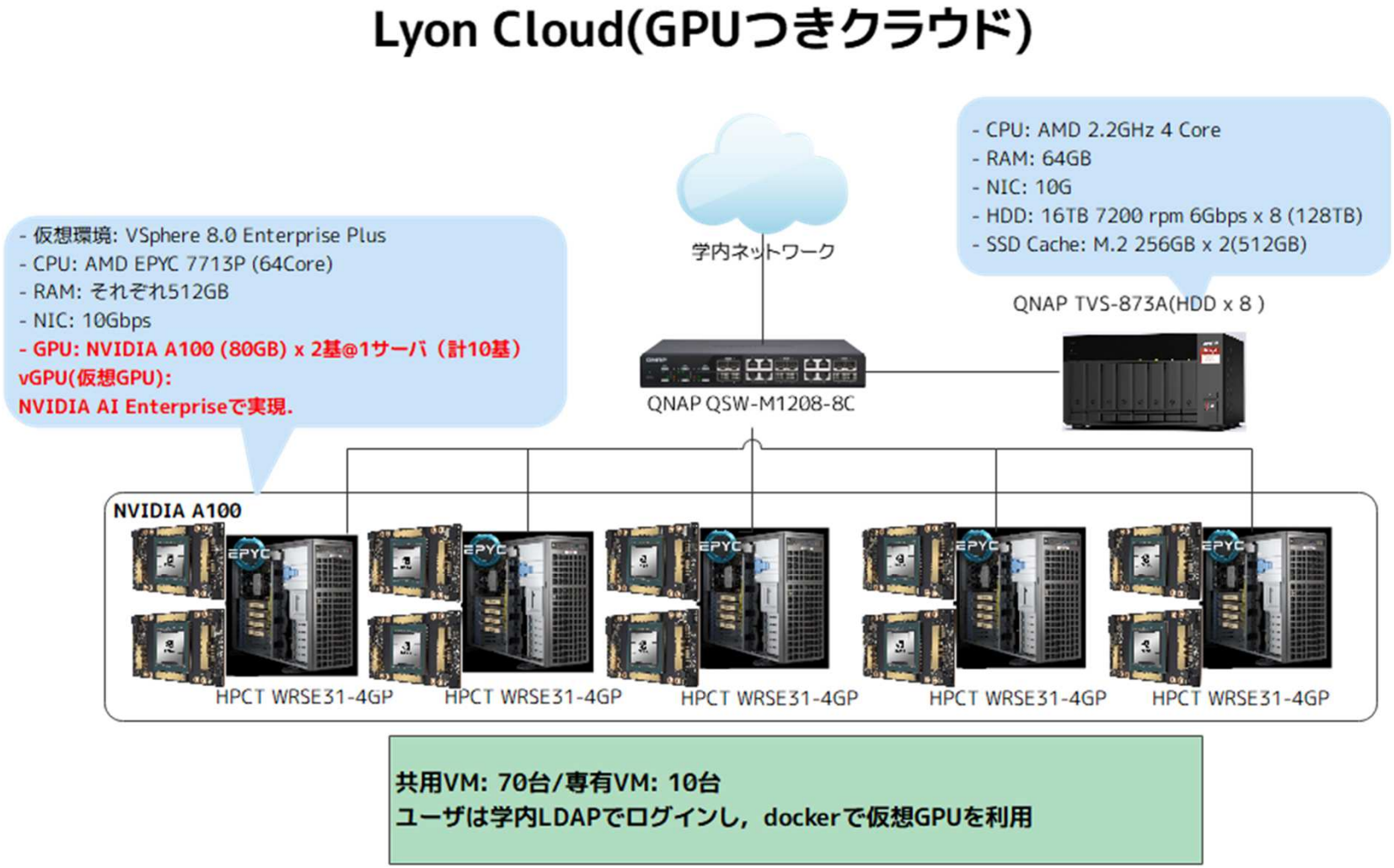
Virtualized Env. for University(Fermi Cloud)

- I implemented in VMWare vSphere.
- Each student can freely create VMs and manage.



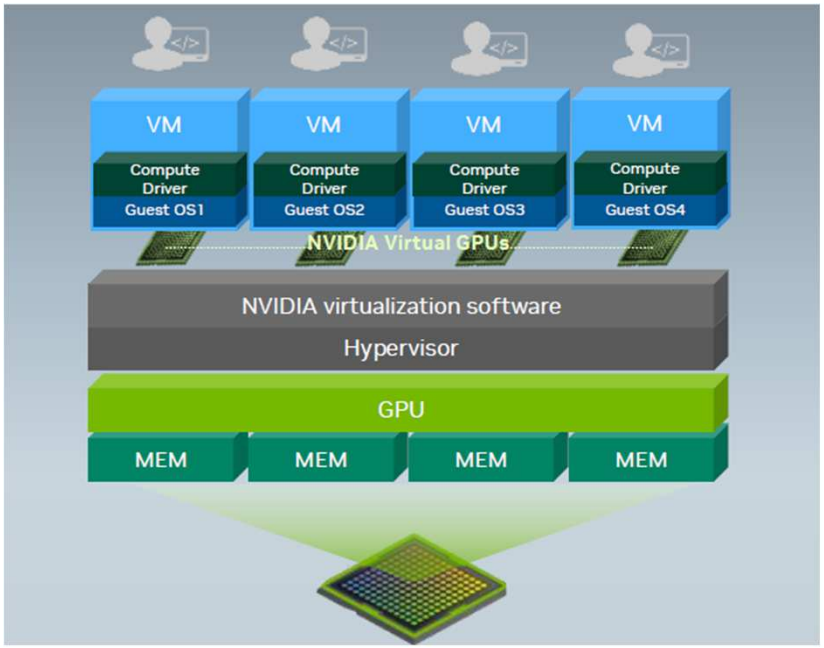
Virtualized AI environment (Lyon Cloud)

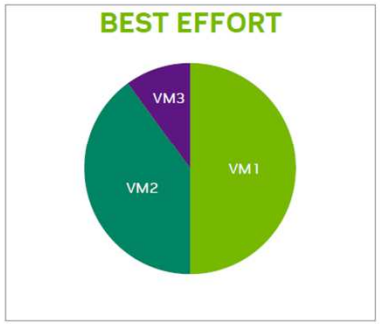
- At most 100 students can freely run machine-learning using GPU simultaneously.
  - Link



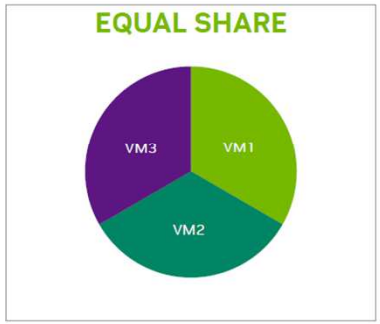
Virtualized GPU/MIG

- We have options: vGPU or MIG (Multi-Instance GPU).

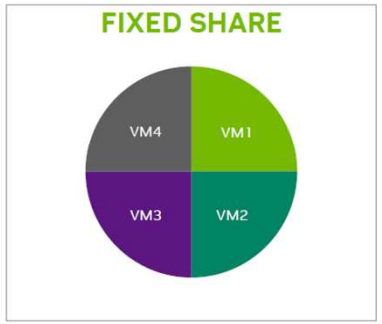




GPUサイクルのシェア
より高いスケールで安定したパフォーマンスを実現し、ユーザーあたりのTCOを削減



GPUのシェア
実行中の各仮想マシンに等しいGPUリソースを提供



GPUのシェア
いつでも同じ品質の定められたサービスを保証

How to use Lyon Cloud

- Each student can login to a VM, and every VM shares “/home” directory.
  - User directory is synchronized.

Lyon VM(共用)の情報

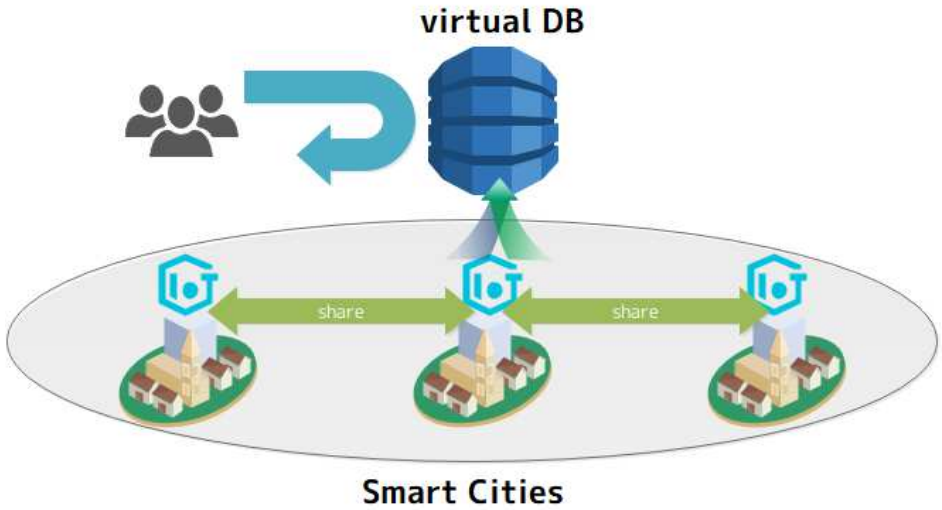
約5分おきにリソース情報を取得しています．ホスト名でsshでログインしてください．
操作方法やコマンドはこちらを参照してください．

ホスト名	vCPU使用率	RAM使用率	ディスク使用率	GPU使用率	GPU RAM使用率
lyon001.cloud.cs.priv.teu.ac.jp	vCPU:1.2%/4vCPUs	RAM:28.9%/16GB	Disk:8.1%/16233GB	GPU:0%	GPU RAM:0.4%/8GB
lyon002.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:11.4%/16GB	Disk:32.5%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon003.cloud.cs.priv.teu.ac.jp	vCPU:0.3%/4vCPUs	RAM:13.3%/16GB	Disk:25.9%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon004.cloud.cs.priv.teu.ac.jp	vCPU:0.3%/4vCPUs	RAM:17.5%/16GB	Disk:13.4%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon005.cloud.cs.priv.teu.ac.jp	vCPU:0.5%/4vCPUs	RAM:12.0%/16GB	Disk:8.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon006.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:12.7%/16GB	Disk:6.0%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon007.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:12.9%/16GB	Disk:20.1%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon008.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:10.7%/16GB	Disk:16.1%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon009.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:13.9%/16GB	Disk:16.2%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon010.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:14.1%/16GB	Disk:31.4%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon011.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:14.6%/16GB	Disk:59.3%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon012.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:14.1%/16GB	Disk:12.6%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon013.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:17.2%/16GB	Disk:33.0%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon014.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:12.4%/16GB	Disk:27.4%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon015.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:14.1%/16GB	Disk:17.7%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon016.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:12.9%/16GB	Disk:28.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon017.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:12.9%/16GB	Disk:71.0%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon049.cloud.cs.priv.teu.ac.jp	vCPU:0.5%/4vCPUs	RAM:12.7%/16GB	Disk:6.9%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon050.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.7%/16GB	Disk:6.5%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon051.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:8.7%/16GB	Disk:6.7%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon052.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.0%/16GB	Disk:6.5%/421GB	GPU:0%	GPU RAM:0.0%/8GB
lyon053.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:11.5%/16GB	Disk:6.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon054.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.4%/16GB	Disk:6.7%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon055.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.3%/16GB	Disk:6.5%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon056.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.5%/16GB	Disk:6.3%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon057.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.4%/16GB	Disk:7.1%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon058.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:13.5%/16GB	Disk:6.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon059.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.1%/16GB	Disk:15.5%/421GB	GPU:0%	GPU RAM:0.0%/8GB
lyon060.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:11.8%/16GB	Disk:13.7%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon061.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:8.5%/16GB	Disk:6.4%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon062.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:9.4%/16GB	Disk:6.6%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon063.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:12.9%/16GB	Disk:6.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon064.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:13.7%/16GB	Disk:6.8%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon065.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:12.7%/16GB	Disk:13.5%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon066.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:9.3%/16GB	Disk:9.6%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon067.cloud.cs.priv.teu.ac.jp	vCPU:0.2%/4vCPUs	RAM:8.4%/16GB	Disk:6.6%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon068.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:15.7%/16GB	Disk:6.8%/421GB	GPU:0%	GPU RAM:0.9%/8GB
lyon069.cloud.cs.priv.teu.ac.jp	vCPU:0.0%/4vCPUs	RAM:8.5%/16GB	Disk:6.6%/421GB	GPU:0%	GPU RAM:0.4%/8GB
lyon070.cloud.cs.priv.teu.ac.jp	vCPU:0.5%/4vCPUs	RAM:8.1%/16GB	Disk:13.7%/421GB	GPU:0%	GPU RAM:0.4%/8GB

National Projects

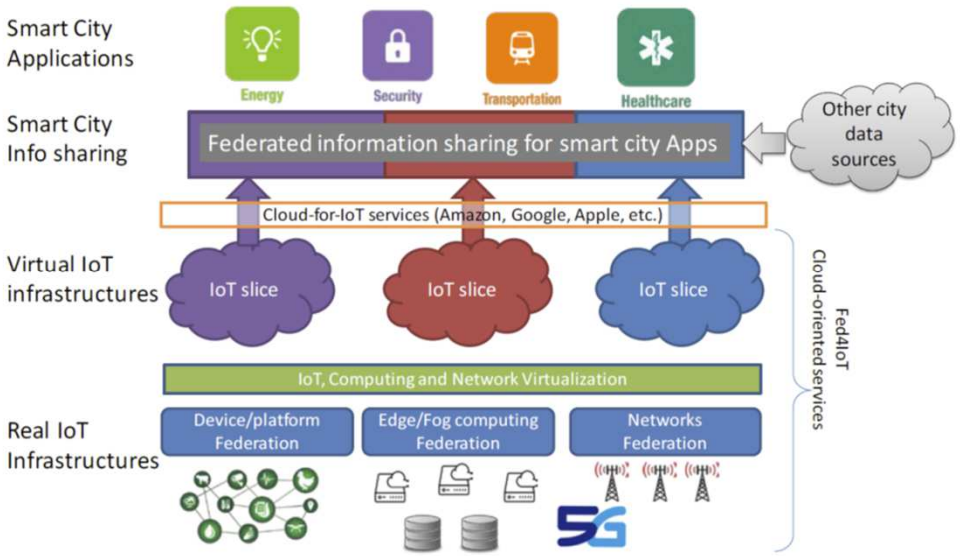
Fed4IoT project

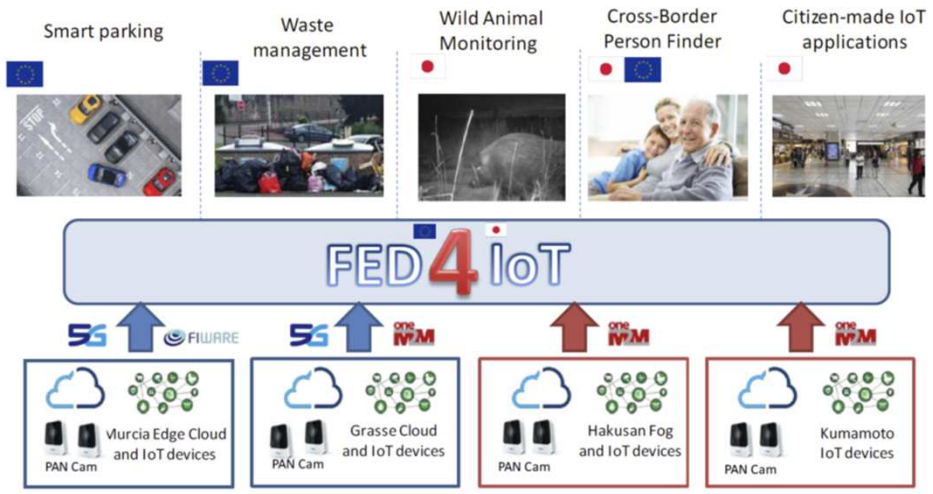
- Current information processing for:
  - IoT data, multimedia streams, large volume file…
- IoT data is sent to heterogeneous systems across regions to share information.
  - E.g., information sharing among **smart cities.** **Fed4IoT project**
- Current systems are virtualized by:
  - Clouds/containers to utilize computational resources.



IoT data sharing in Fed4IoT

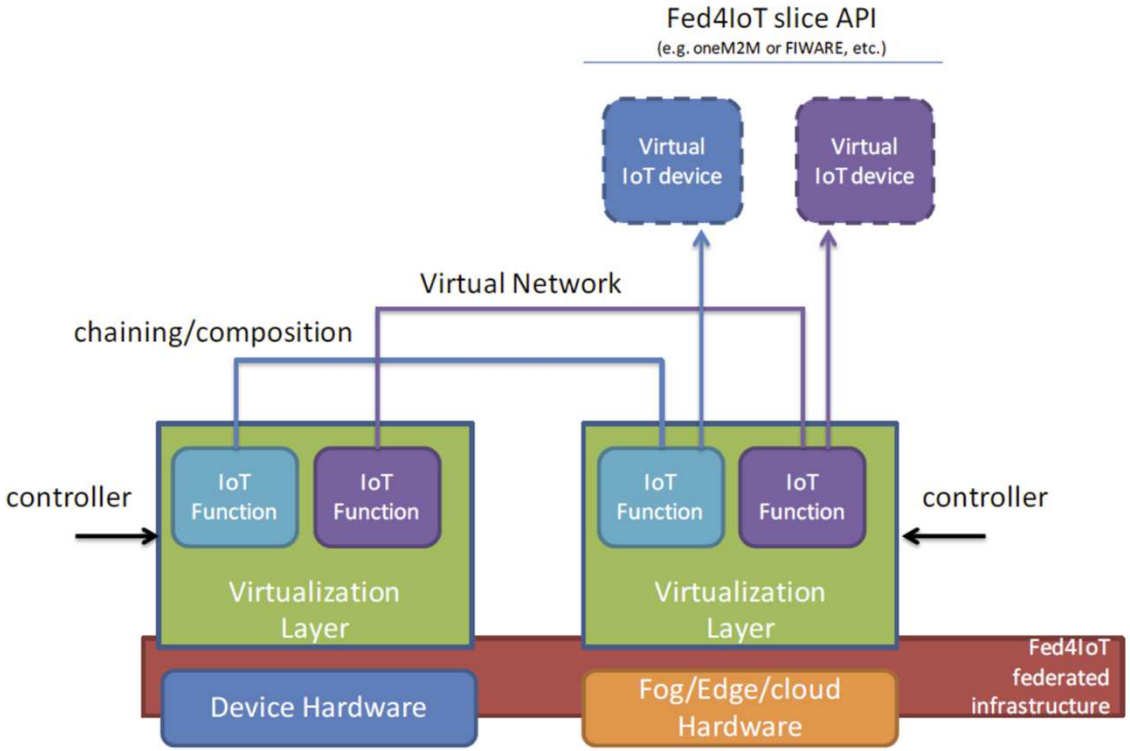
- The key idea for utilizing IoT data is to share it among regions.
- Then each IoT device is virtualized to generate "virtual thing"
  - E.g., to provide the averaged temperature among EU/JP transparently from users.





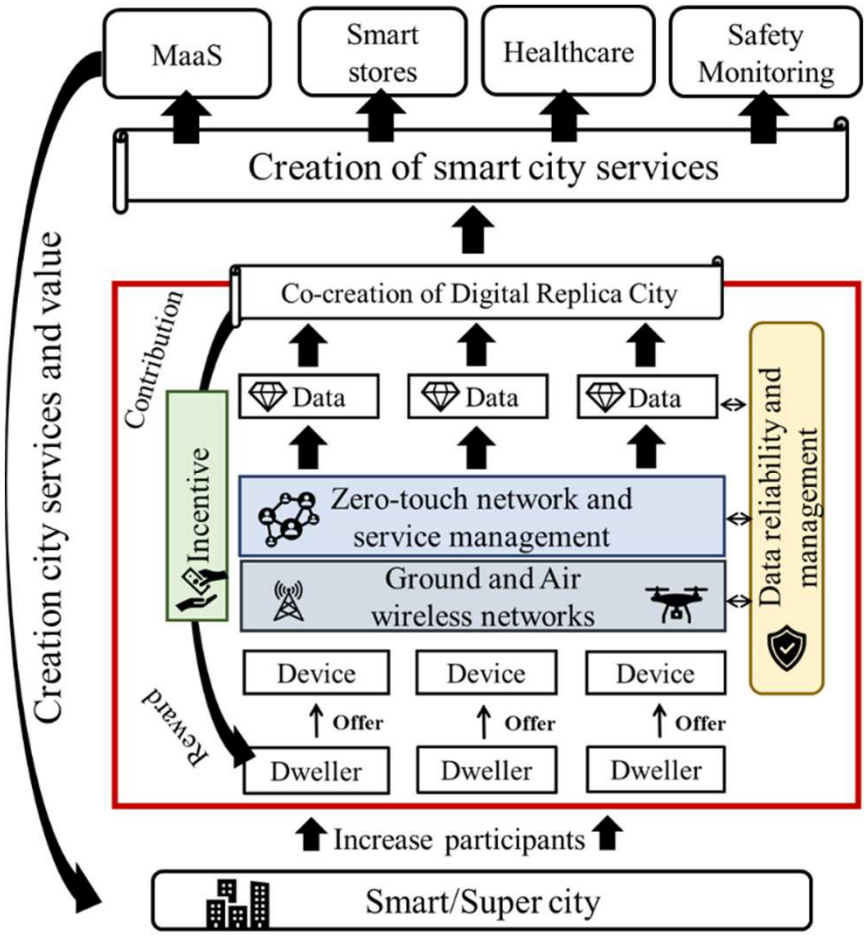
What kinds of data should be shared?

- Processing results as well as sensed data.
  - Chaining functions and executes to generate “virtual data”.
- Each function is expressed as container.
  - Pods in Kubernetes, docker containers….



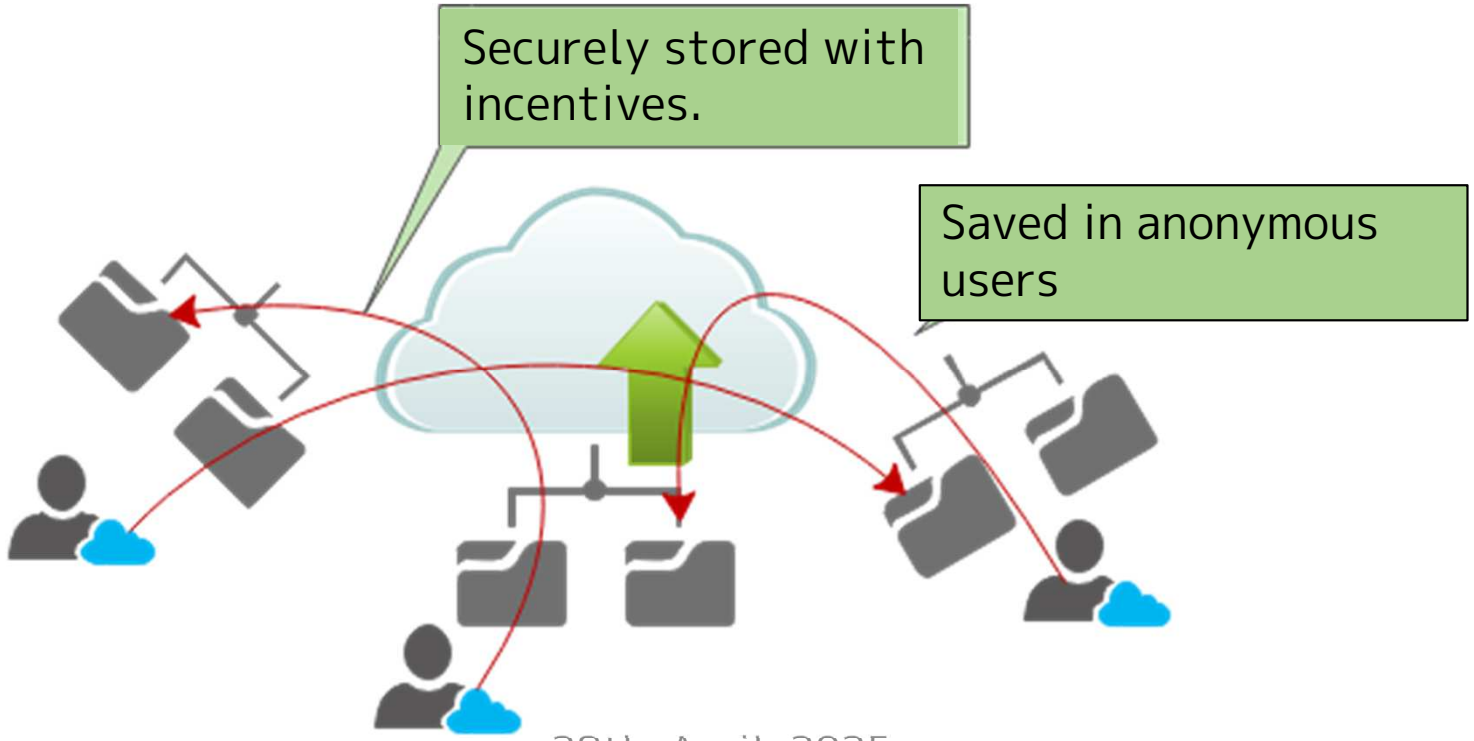
D2ecosys: Co-creating Digital Twin

- D2EcoSys: decentralized digital twin ecosystem.
- We realized an incentive model to make people contribute to create digital twins.



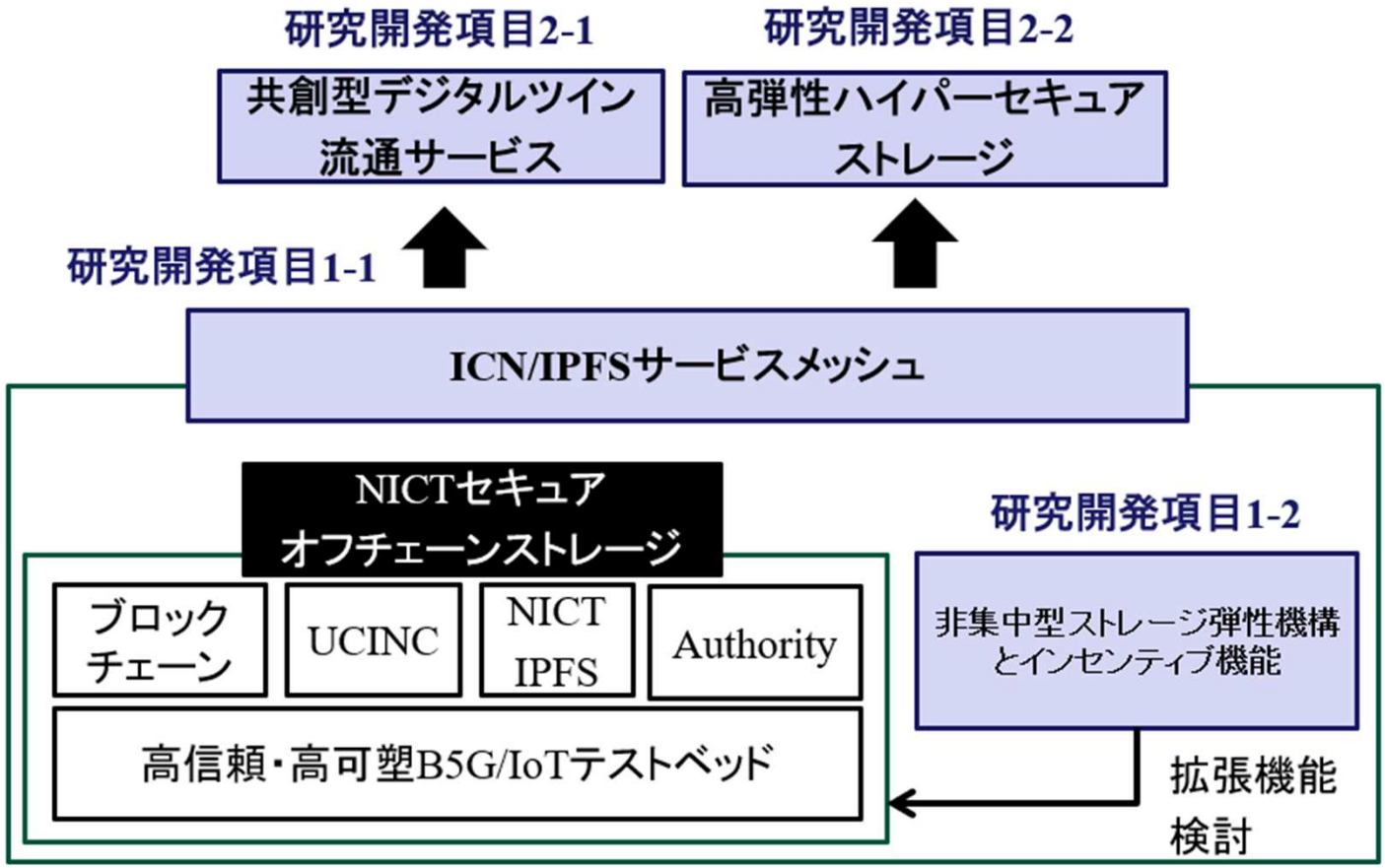
Project with Protocol labs.

- IPFS(Interplanetary File System) is a content-centric P2P middleware for distributed database from Protocol Labs. Inc.
- Every file can be deployed / stored on nodes around the world in a distributed manner.
- We realized a lookup performance in IPFS.

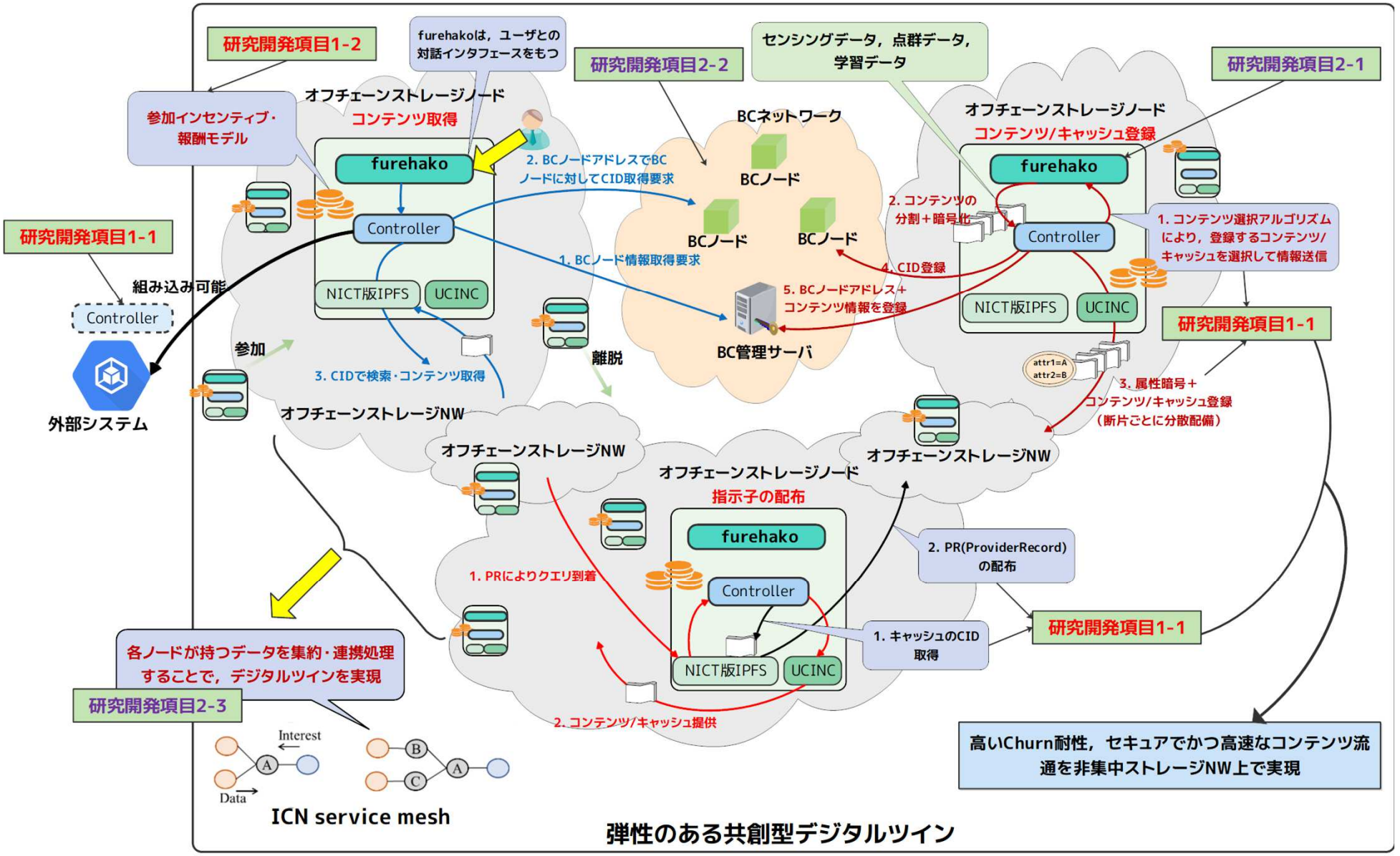


Web3-enabled digital twin/distributed DB

- Our proposal has been accepted as a national project by NICT, starting from April. 2025.



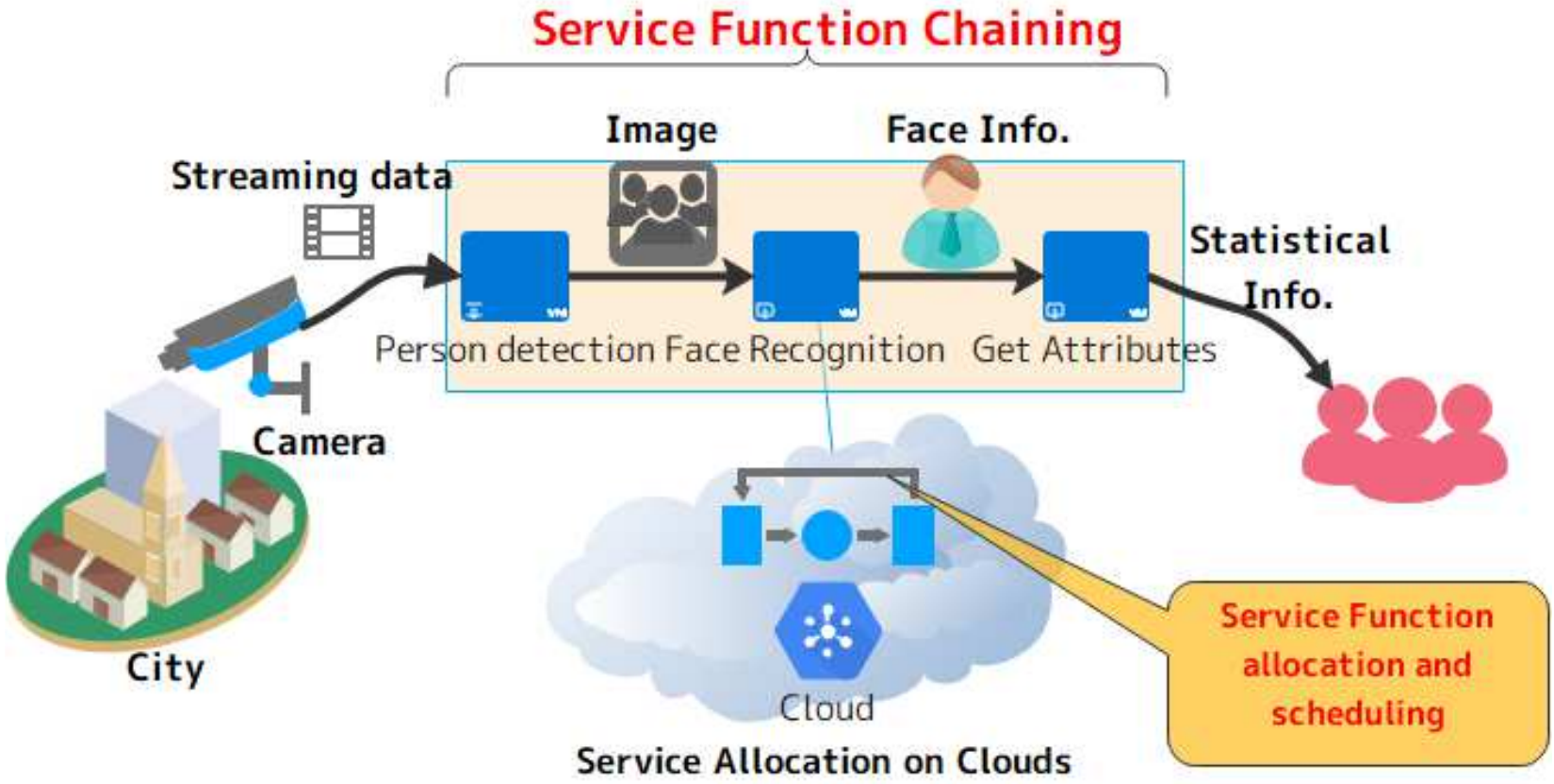
Detailed processes in this project.



Involved Technologies

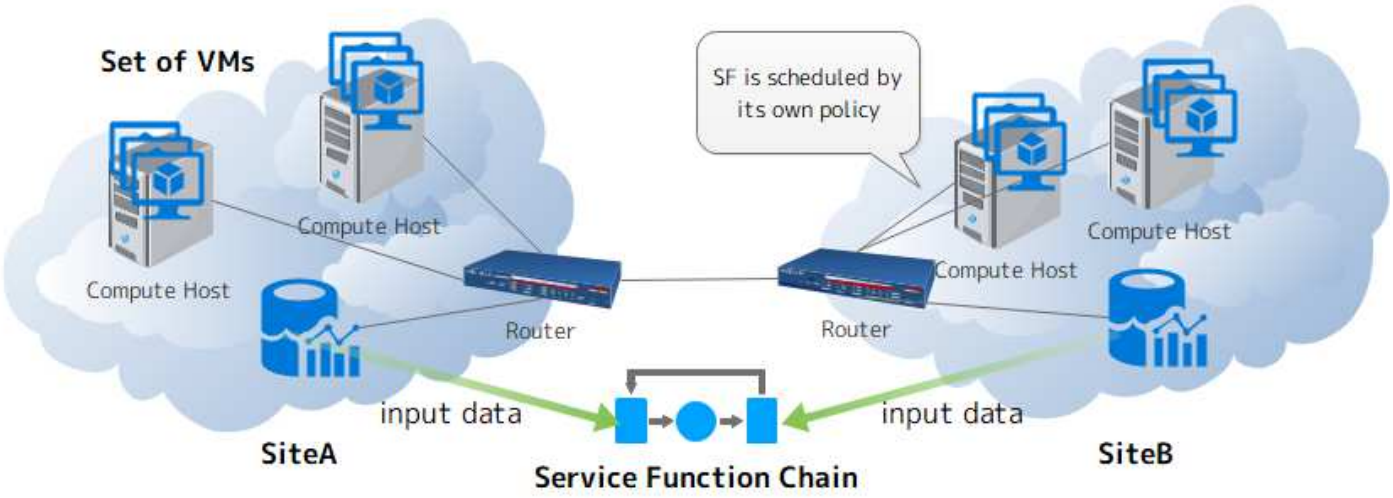
IoT data processing on clouds

- Efficient service function allocation/scheduling algorithms for IoT applications.
  - A service function: virtualized instance on a cloud.



Motivation

- Resource data should be stored among regions.
  - Smart city, human faces for person finder, sensed data…
- **Cross-border SFC** is needed if each site's data is utilized for collaboratively process SFs.
- Scheduling/Allocation policy depends on each area.
  - **Autonomous/decentralized scheduling is required.**

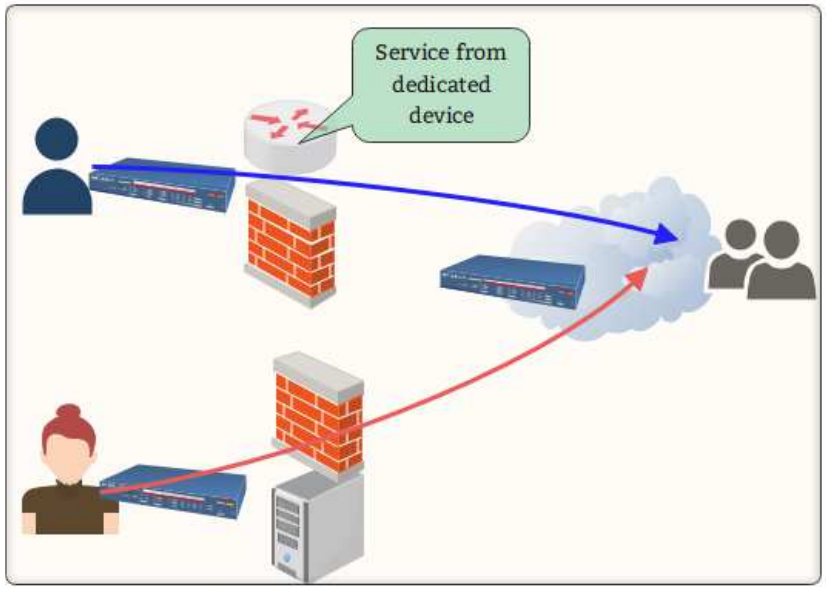


Keywords in my research

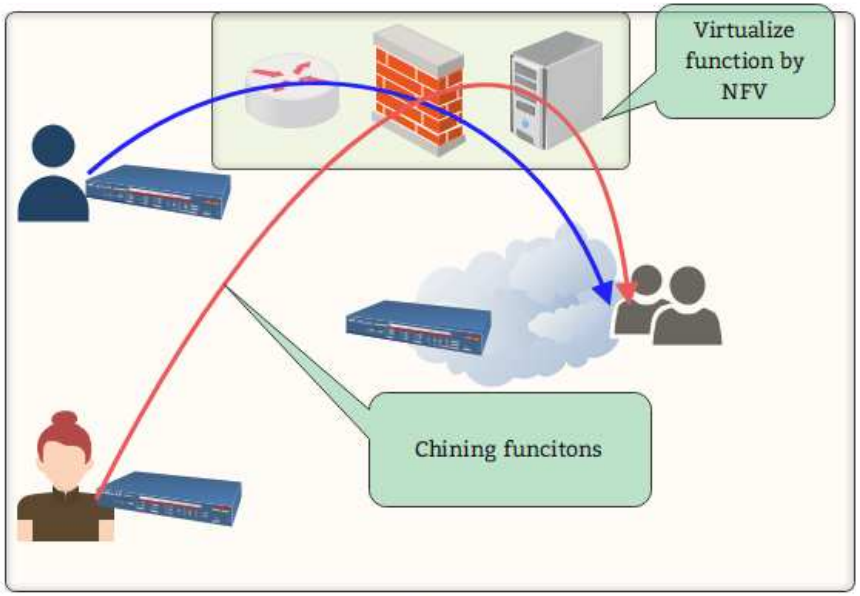
- Dockerized Service Function allocation
- Workflow Scheduling (Task Scheduling)
  - In centralized/decentralized manners
- ICN (Information-Centric Networking)
  - For decentralized workflow scheduling.
  - Utilizing “caching scheme” for efficient parallel execution.
- DHT (Distributed Hash Table) for IPFS

Function chaining

- NFV (Network Function Virtualization) is to chain functions to provide a large service.
  - E.g., VNF1(firewall) ->VNF2(routing) ->VNF3(load balance)
  - Data communication is done among functions.
- One of generalized forms is “workflow”.



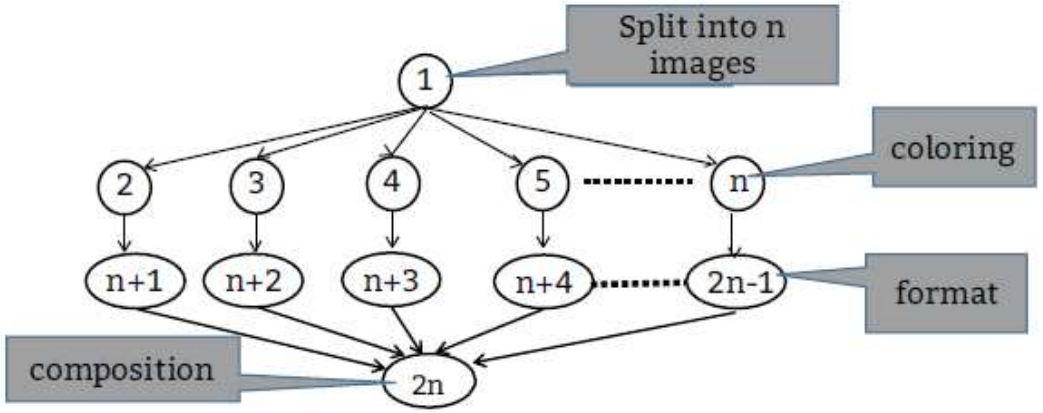
Conventional network services

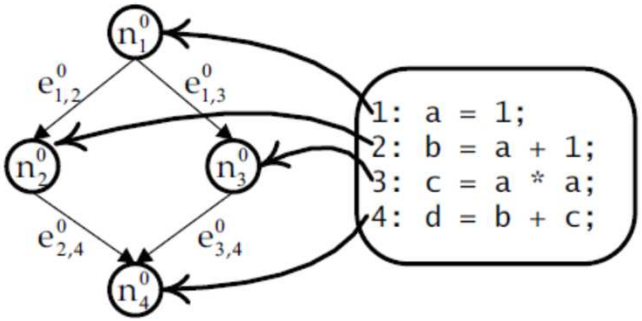


NFV

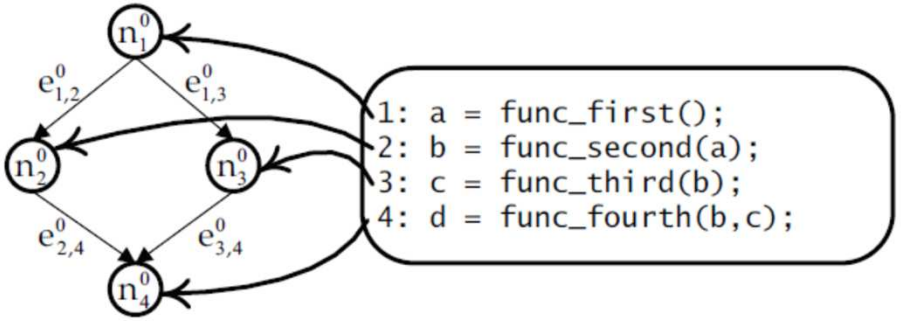
What is “workflow”?

- Each “task” has data dependencies with others, i.e., precedence constraint.
- Task: an “atomic” process such as a container.
- General form of a job.





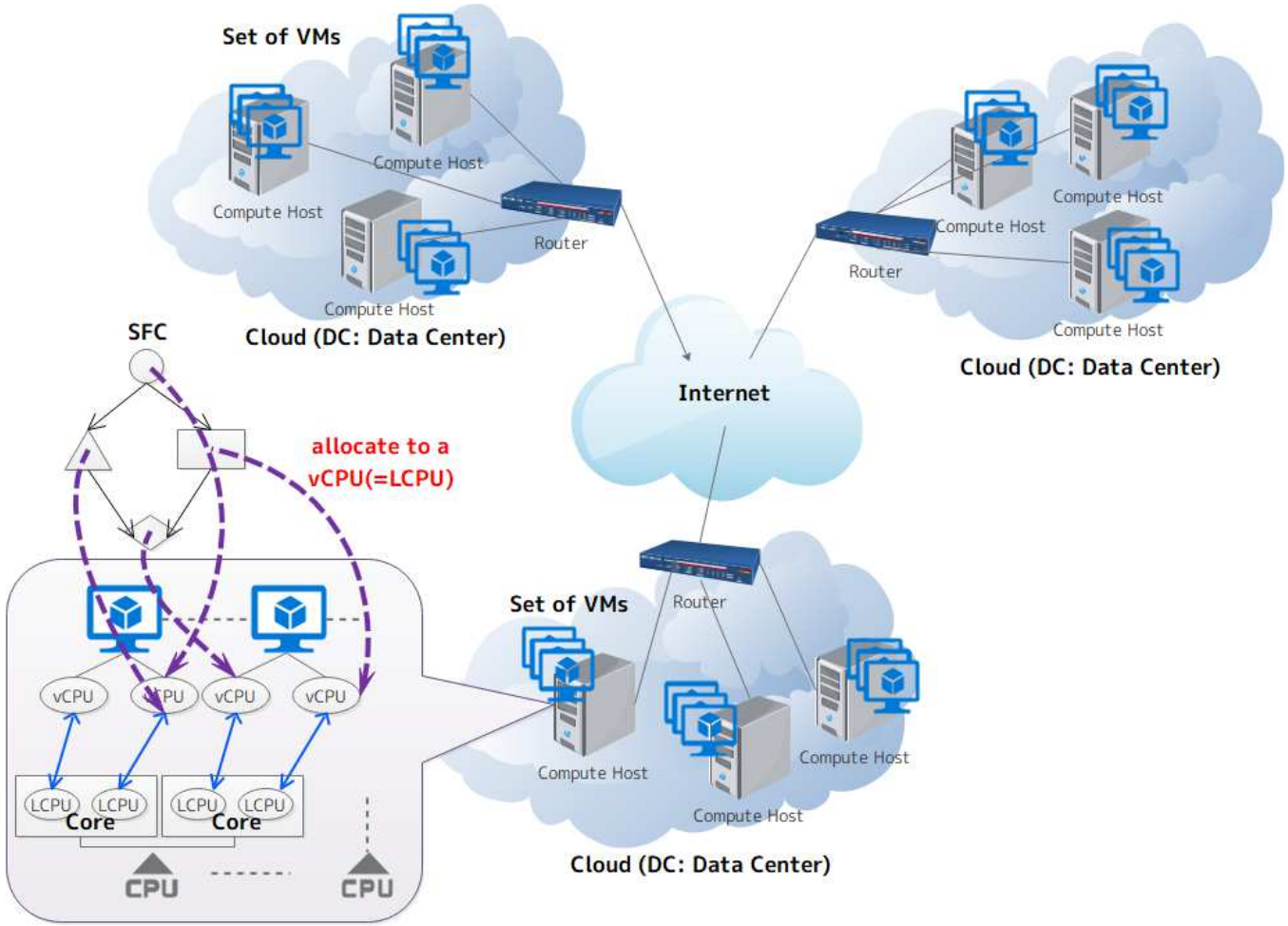
(a) 1 task :1 statement



(b) 1 task :1 function call

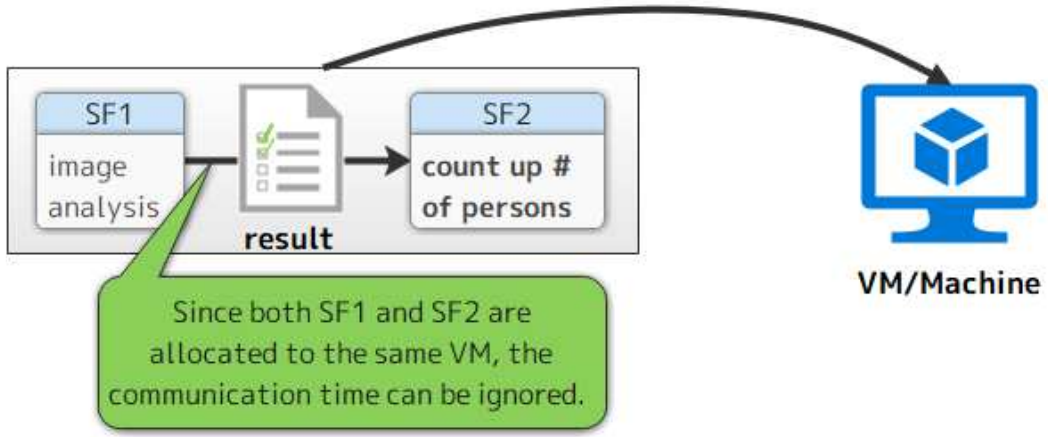
Assumed environment

- Each area has its own cloud(s) that has dedicated data for private data sharing.
- Cross-border data sharing for collaborative processing.



Scheduling properties

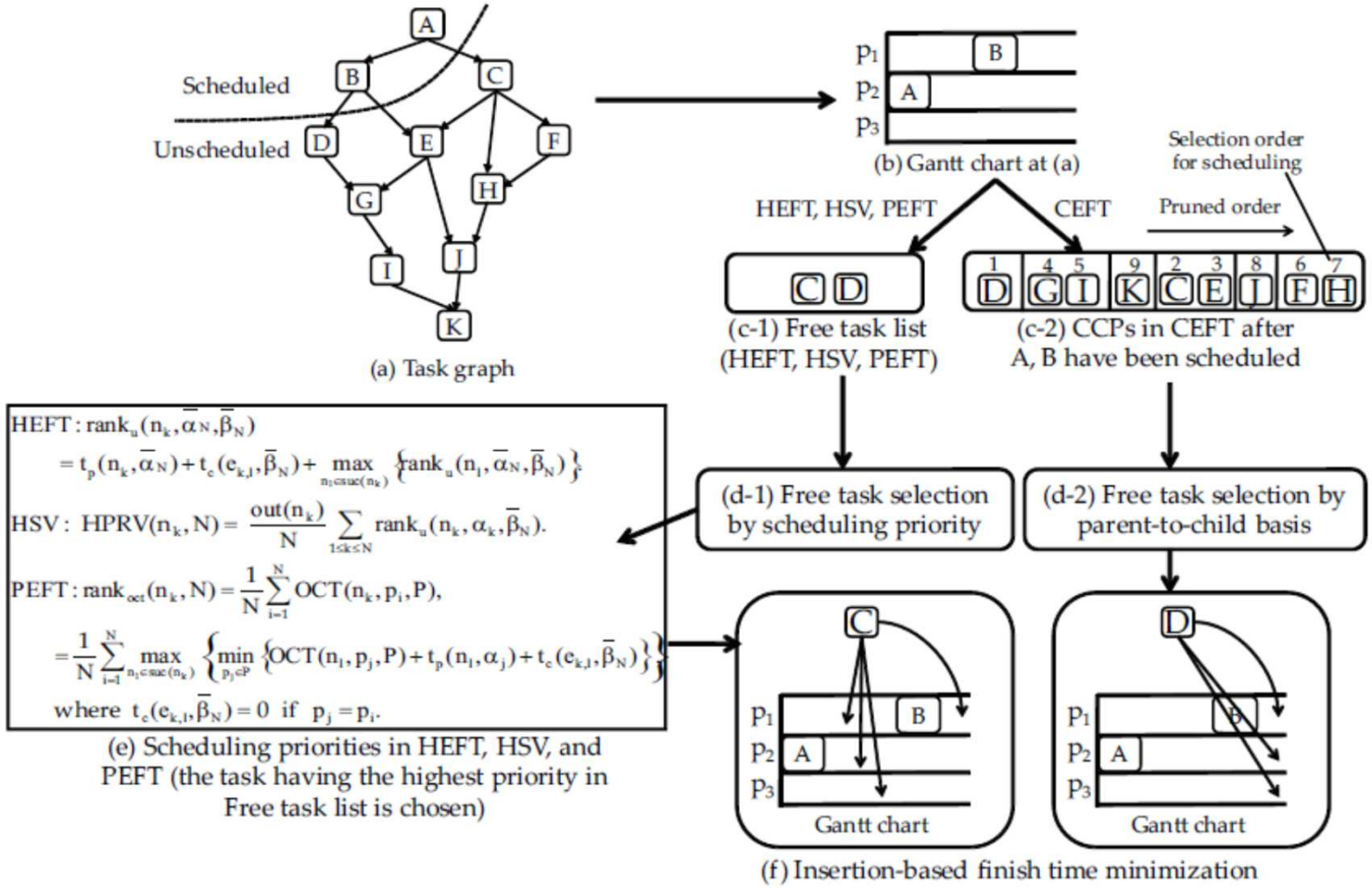
- Communication time can be localized depending on the allocation targets.



- “scheduling” : allocation and ordering.
  - List-scheduling
  - Clustering

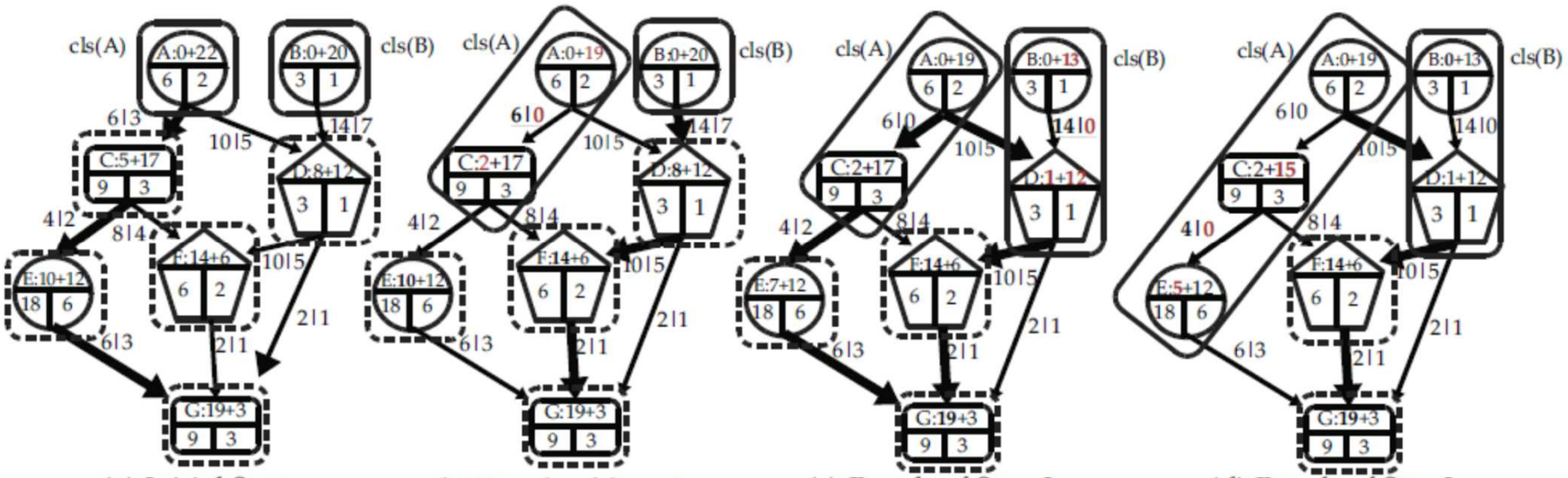
List scheduling

- Task selection phase (by a specific priority)
- Task allocation phase (to minimize the makespan)



Task clustering

- Clustering several tasks into a larger one in order to localize the communication among them.
- As a result, the makespan can be effectively for data-intensive jobs.
- The balance between parallelism and data localization should be considered.



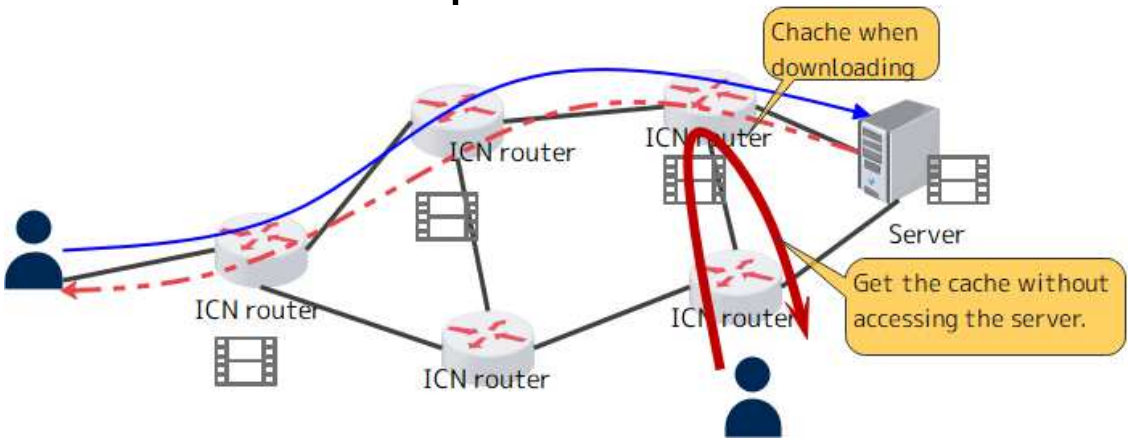
Scheduling for container-based jobs

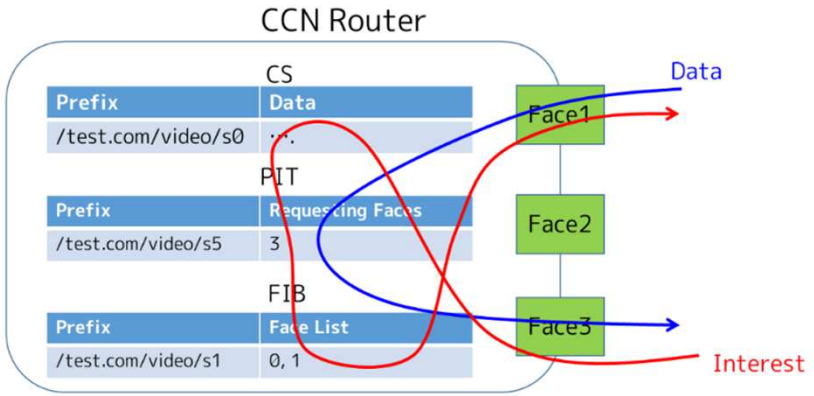
- Conventional scheduling algorithm (list-based/clustering-based) cannot be directly applied to container-scheduling.
  - Delay for pulling containers should be considered.
  - One container can be shared among tasks.

- **Idea for reducing the # of pulling containers can leads to the active container sharing.**

ICN (Information-centric networking)

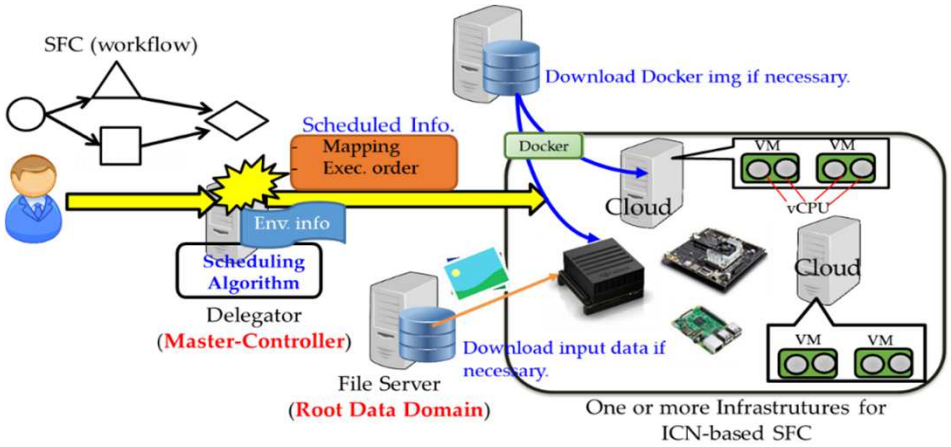
- In contrast to IP network, each content is accessed by “content name”.
  - **Users do not need to aware of the location.**
- Each ICN router has:
  - CS (Content Store): Cache DB
  - PIT (Pending Interest Table): Routing table for returned data.
  - FIB (Forwarding Information Base): Routing table for interest packets.



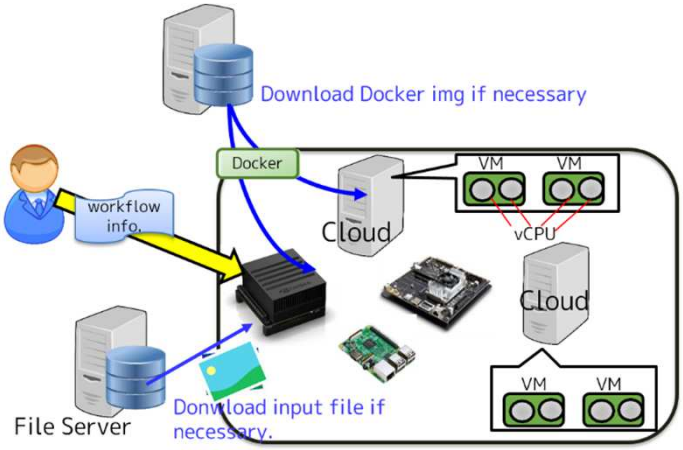


Centralized vs Decentralized approaches for scheduling SFs

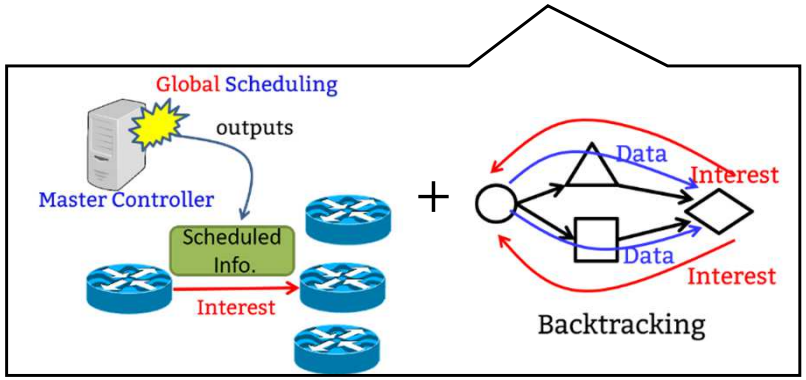
- If container scheduling is performed within one site, “centralized scheduling” is appropriate for optimization. Otherwise, decentralized approach is needed.

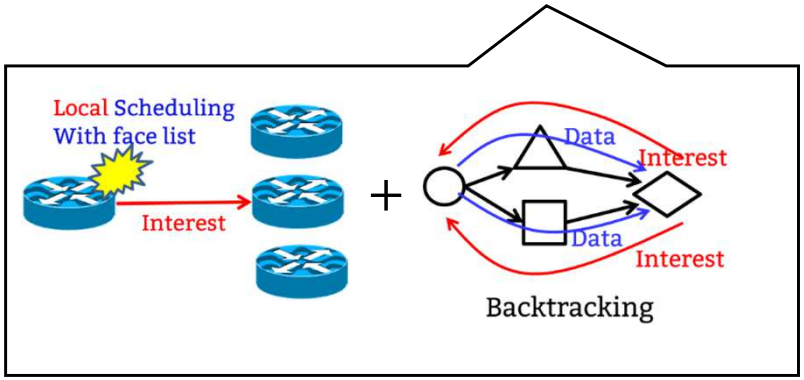


IP/ICN-SFC with master



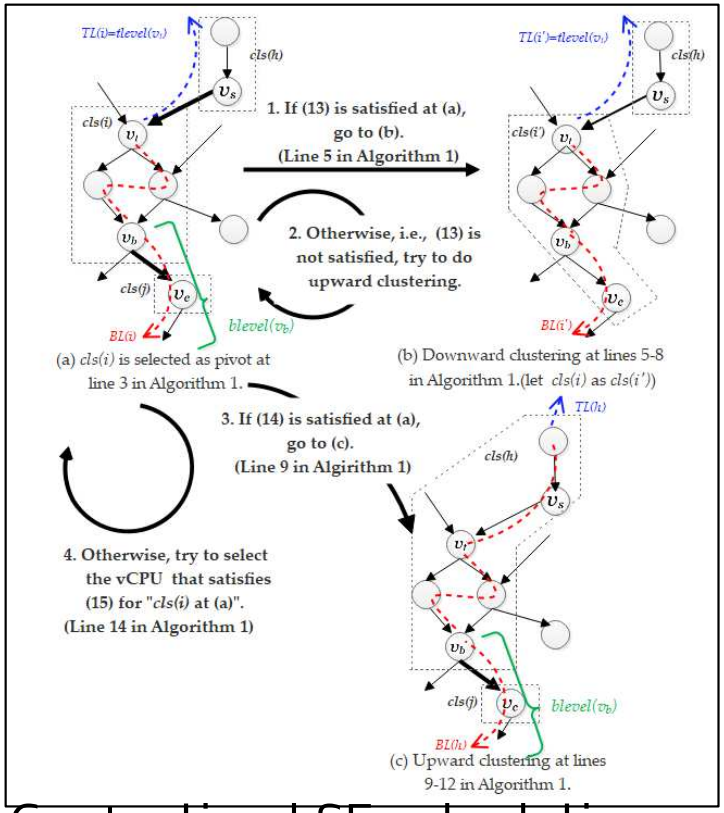
Full-Distributed ICN-SFC
(Without master)



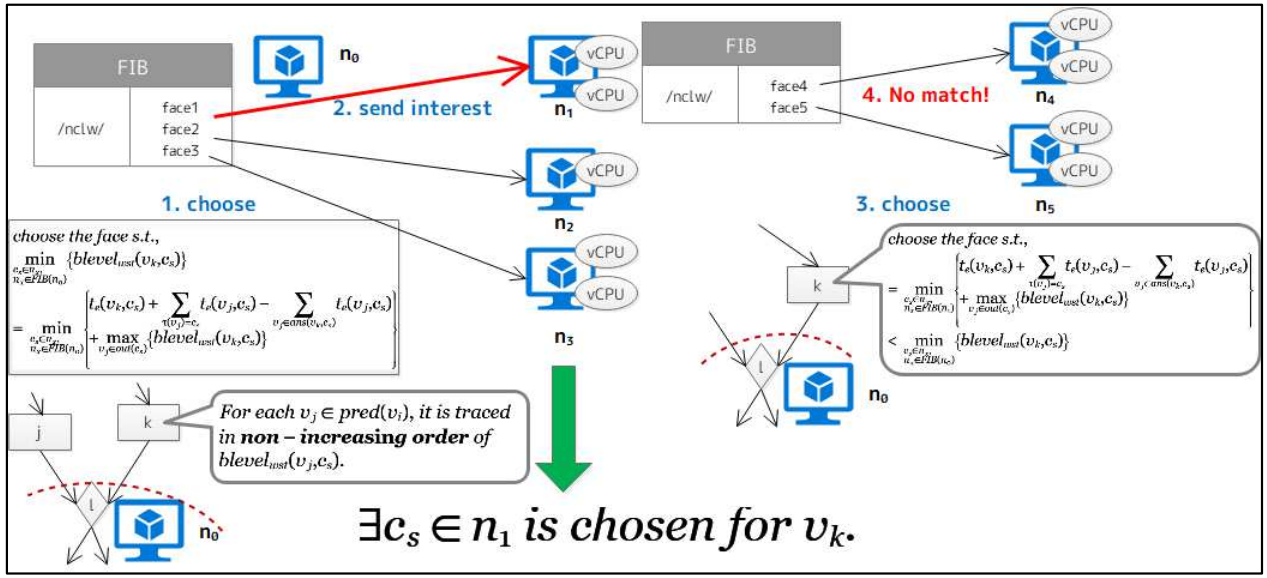


Our developed scheduling algorithms

- We developed **workflow-type SFC scheduling algorithms**.
  - IP-based Centralized-based one.
  - **decentralized-based with ICN (each node schedules autonomously)**



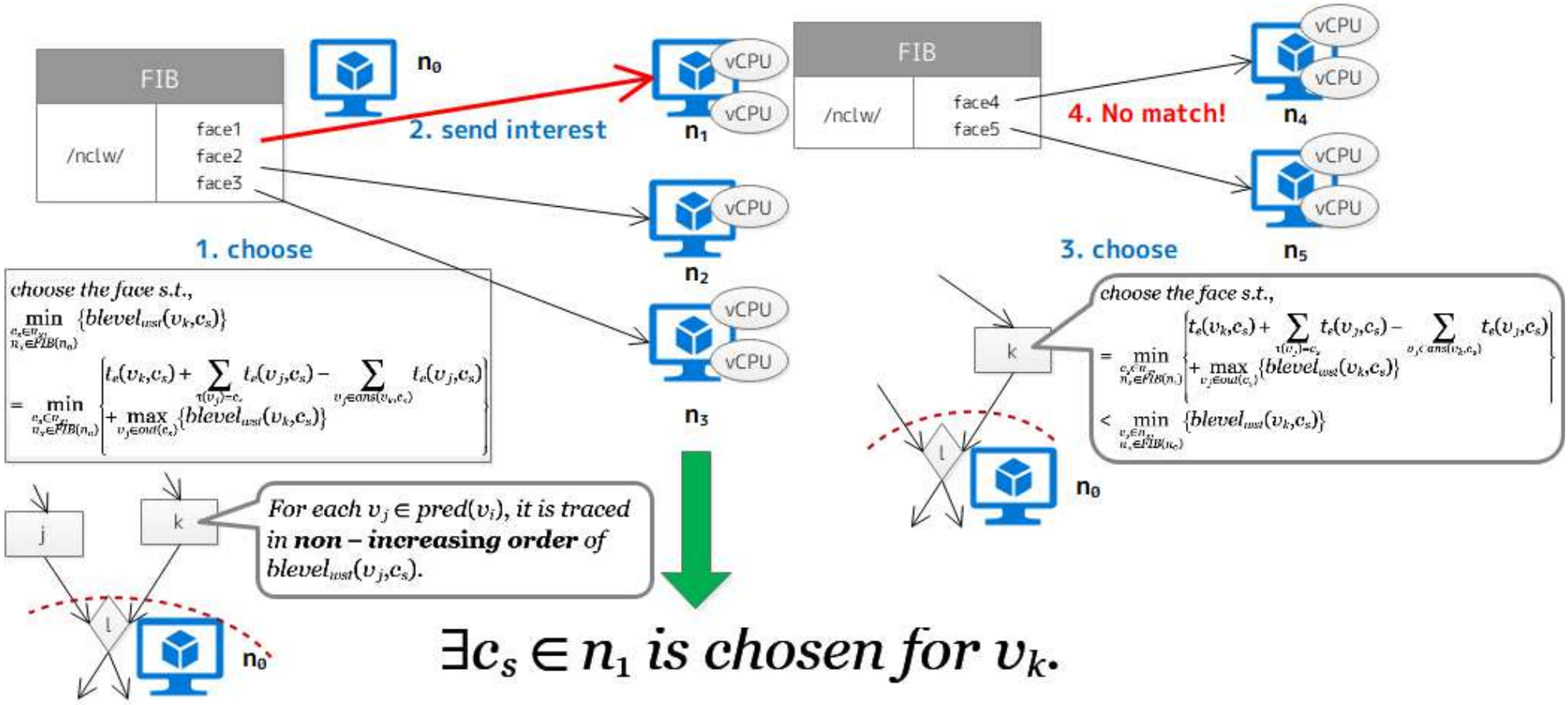
Centralized SF scheduling



ICN-based decentralized scheduling

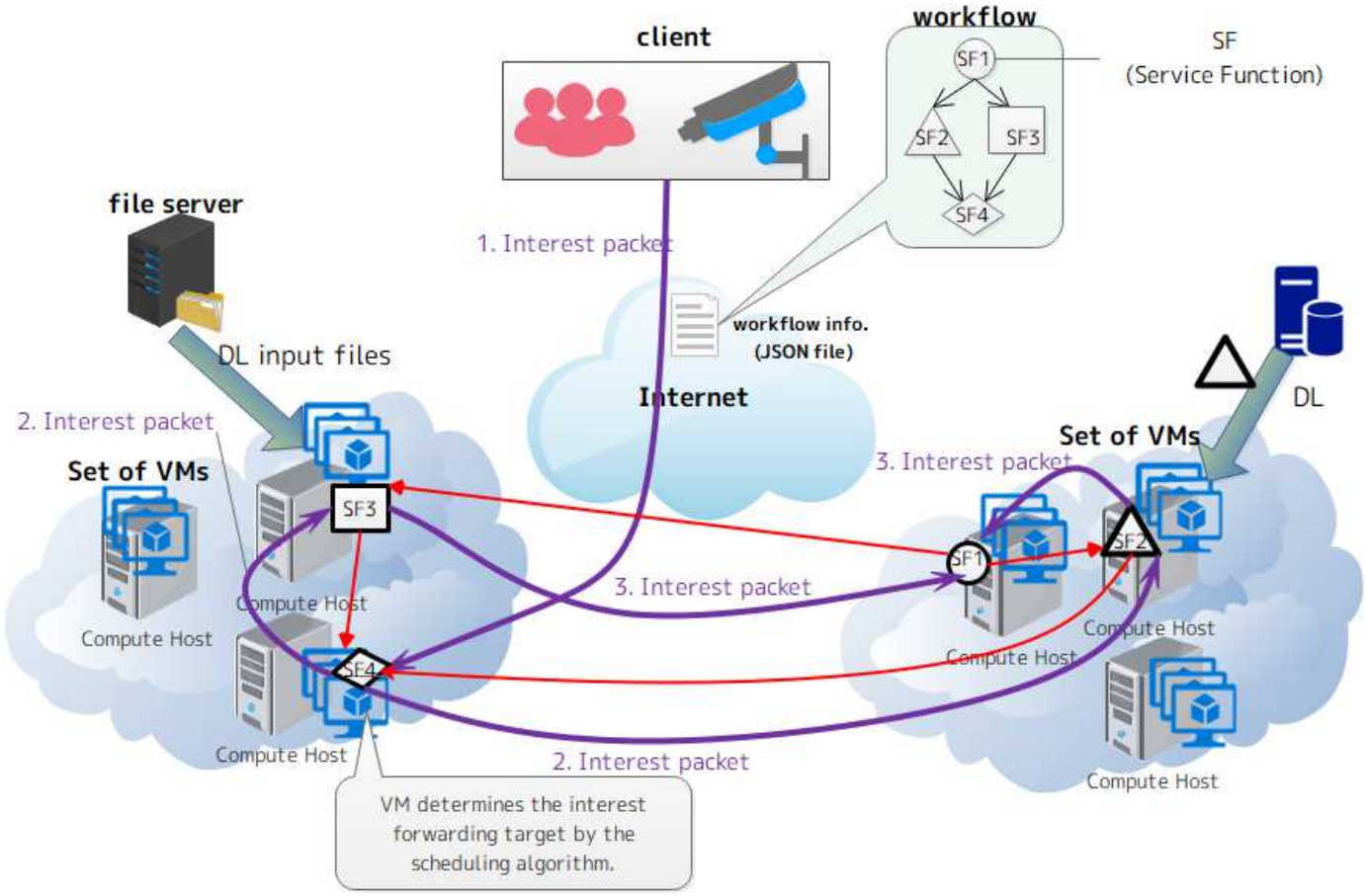
ICN-based decentralized scheduling

- Each node tries to **select the allocation target node from its own FIB** for the predecessor task.
  - The node that minimizes the slack time is selected.



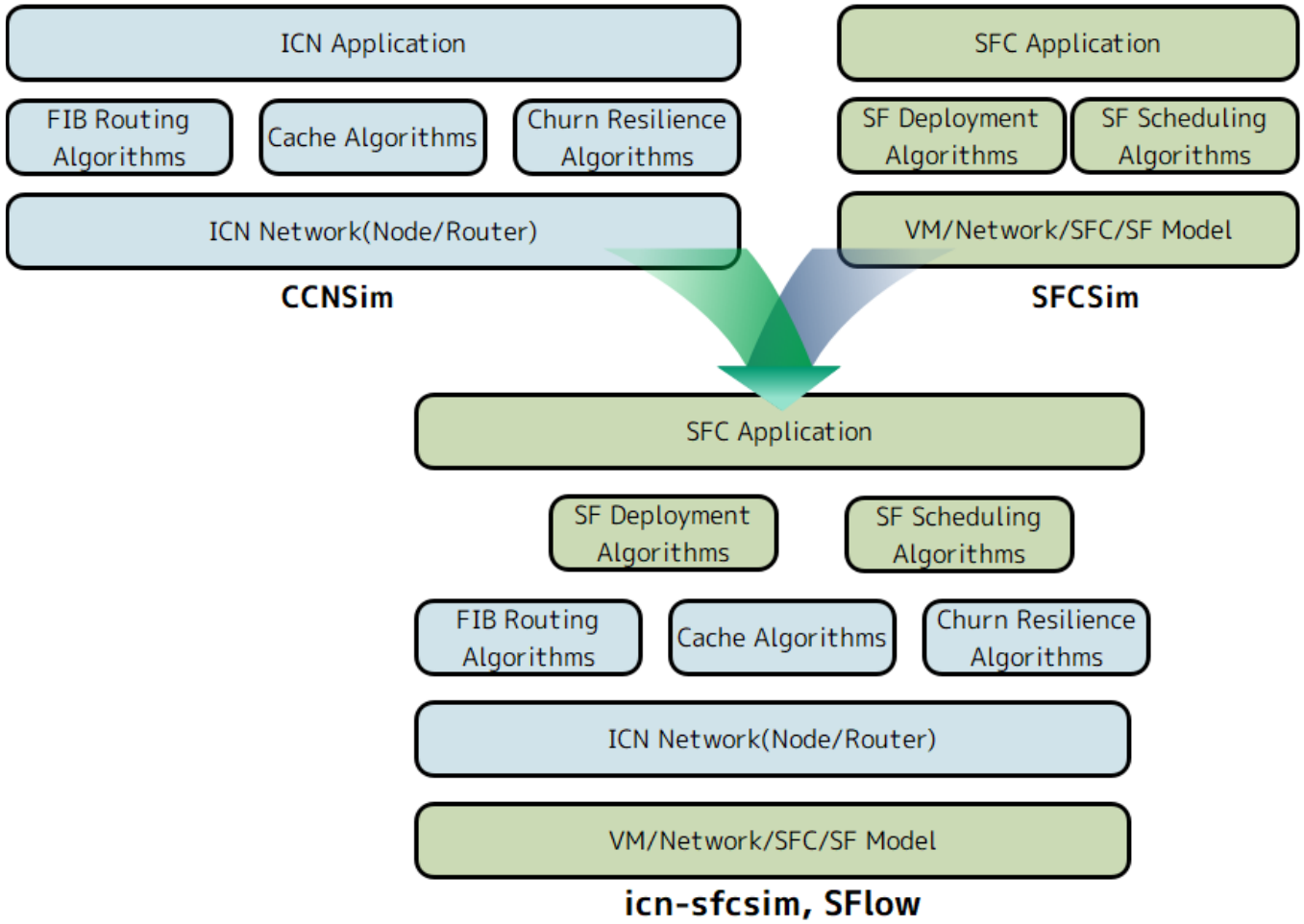
Why ICN is used for SFC?

- To handle data mobility, efficient processing with data caching.
  - Location-based data addressing is not suitable for cross border processing.



Evaluation environment

- We are conducting experimental comparisons by:
  - Simulation (icn-sfcsim)
  - Real environment(SFlow)



Simulation software

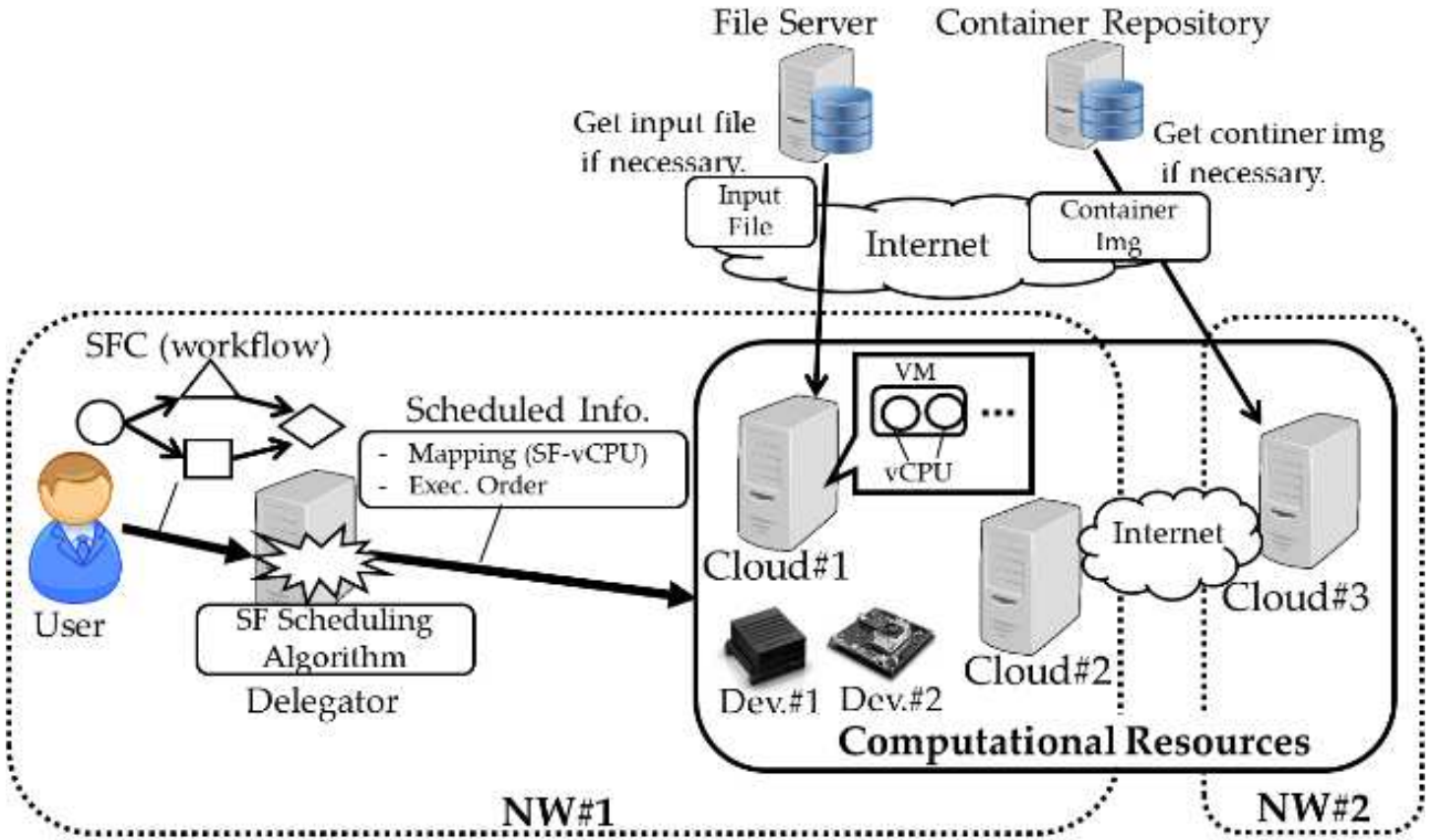
- We developed **icn-sfcsim** that performs:
  - https://github.com/ncl-teu/ncl_icn-sfcsim
  - Event-driven simulation for allocating SF->node
  - Scheduling is performed on the simulated ICN network
    - ICN is based on **ccnsim** that we developed on GitHub.
      - ccnsim URL: https://github.com/ncl-teu/ncl_ccnsim
    - SF Scheduling is based on sfcsim that we developed on GitHub
      - sfcsim URL: https://github.com/ncl-teu/ncl_sfcsim

Workflow engine with IP/ICN

- **SFlow**(https://github.com/ncl-teu/SFlow) is the workflow engine for task/function scheduling in a workflow.
- SFlow supports several workflow scheduling algorithms, and we can switch the one by the config. File.
- Supported chaining schemes:
  - IP-based (Master-worker based scheduling)
  - ICN-based (implemented with jNDN)
    - Master-worker based scheduling
    - Decentralized scheduling

Example: Real environment

- Our developed scheduling algorithms are performed in the following environment.
- Of course, this can be extended for scale up.



Specification

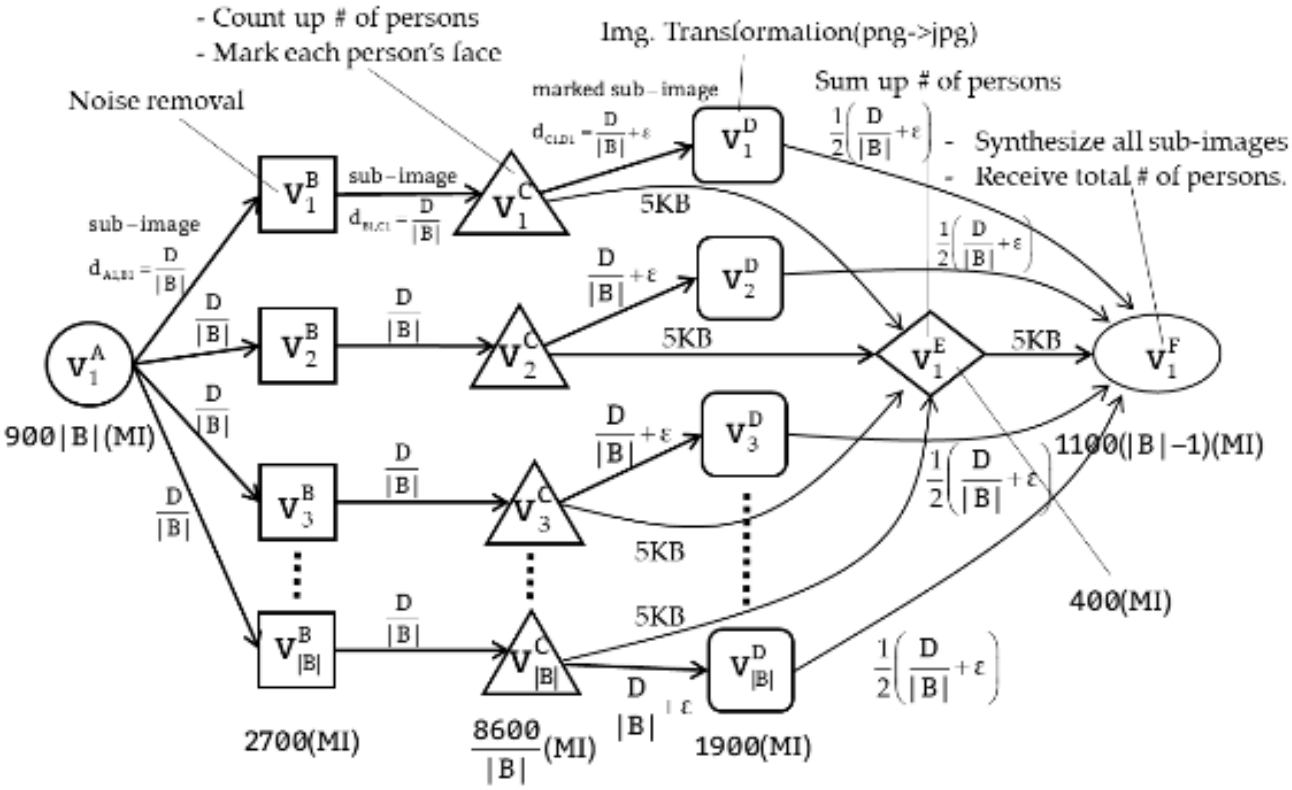
- Specs. Of the real environment.

Hardware	Parameter	Value
NW#1 Router#1	Bottleneck BW to external hosts	100Mbps.
	Internal BW	1Gbps
Cloud#1	# of VMs	15VMs
	# of vCPUs	2vCPUs x 4VMs 4vCPUs x 11VMs
	vCPU Freq.	{2.0, 2.5, 3.0} GHz.
	BW to Router #1	1Gbps
Cloud#2	# of VMs	6VMs
	# of vCPUs	2vCPUs x 4VMs 4vCPUs x 2VMs
	vCPU Freq.	{1.0,2.0, 2.5} GHz.
	BW to Router #1	1Gbps
Dev.#1	Type	NVIDIA Jetson AGX Xavier (2.26 GHz x 8Cores, 16GB RAM)
Dev.#2	Type	NVIDIA Jetson TX2 (2.0 GHz x 6Cores, 8GB RAM)

Hardware	Parameter	Value
NW#2 Router#2	Bottleneck BW to external hosts	100Mbps.
	Internal BW	1Gbps
Cloud#3	# of VMs	8VMs
	# of vCPUs	4vCPUs x 4VMs 4vCPUs x 2VMs
	vCPU Freq.	{1.0, 3.0, 3.5} GHz.
	BW to Router #2	1Gbps

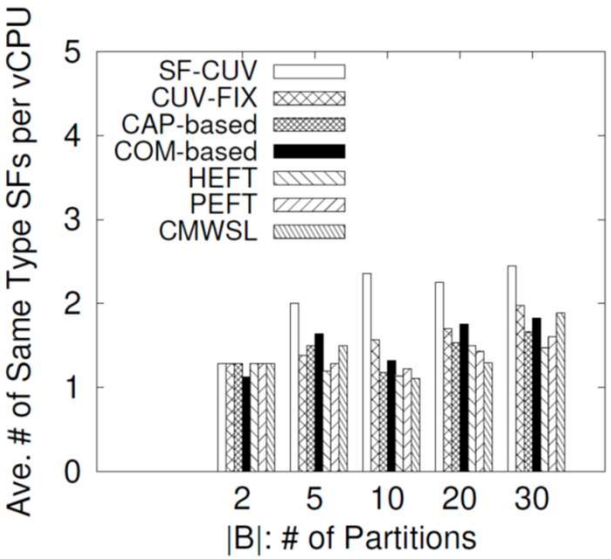
Tested application

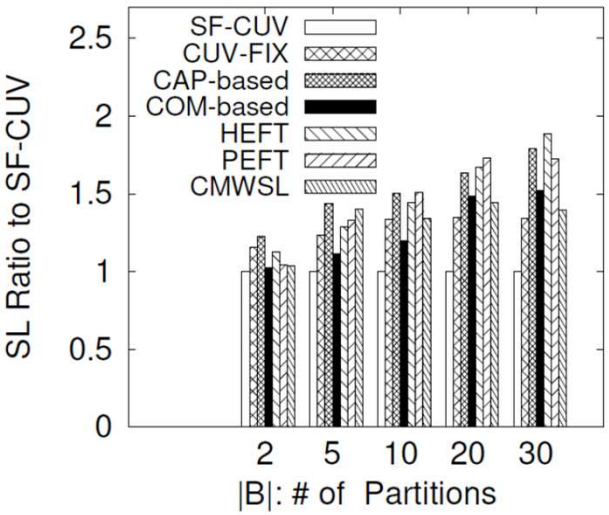
- Dockerized SF is prepared and scheduled:
  - Image processing that count up # of persons for an image.

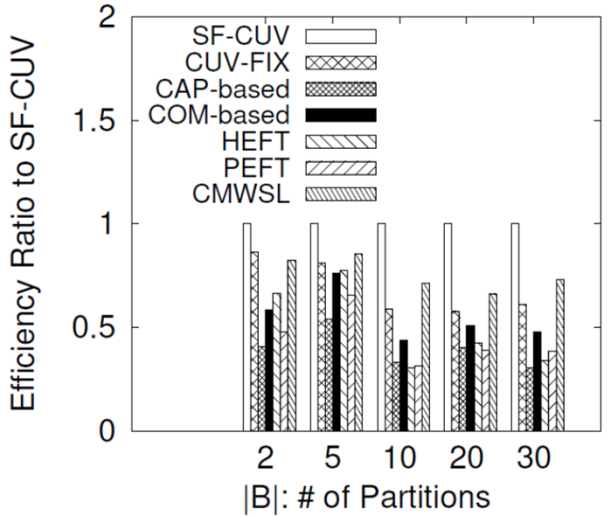


Comparison of no SF pre-deployment

- Our proposal: “SF-CUV (Service Function Clustering for Utilizing vCPUs)”.
- SF-CUV outperforms others in terms of:
  - Makespan (Schedule length)
  - # of required SF instances.
  - Efficienty

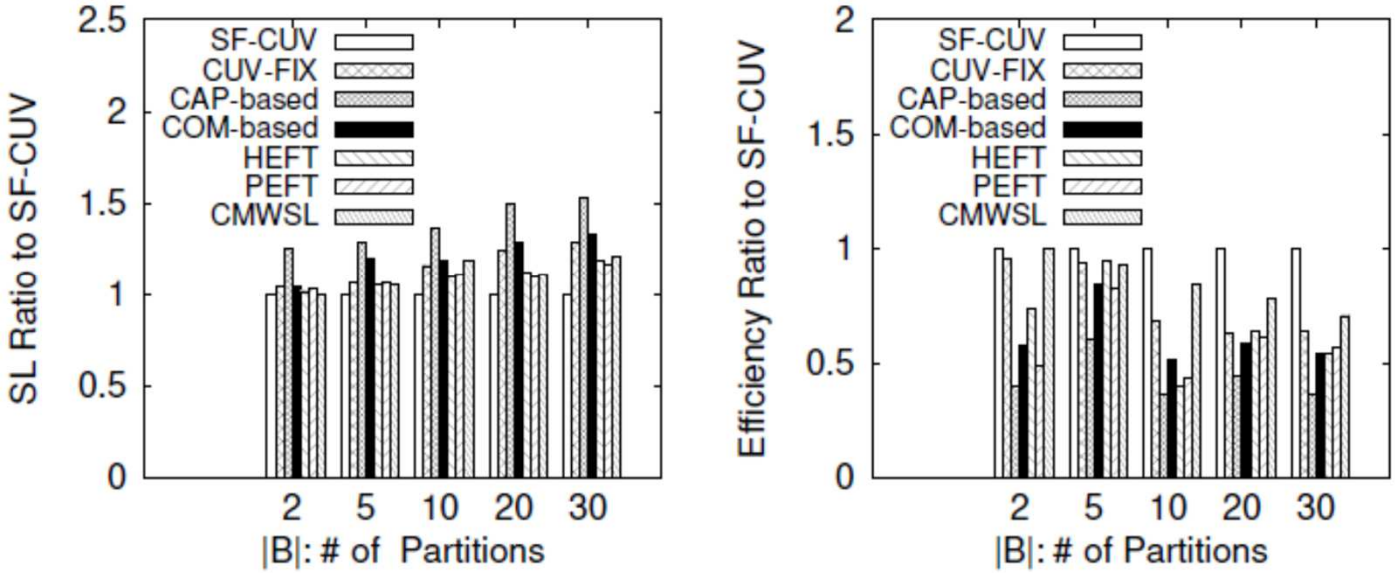






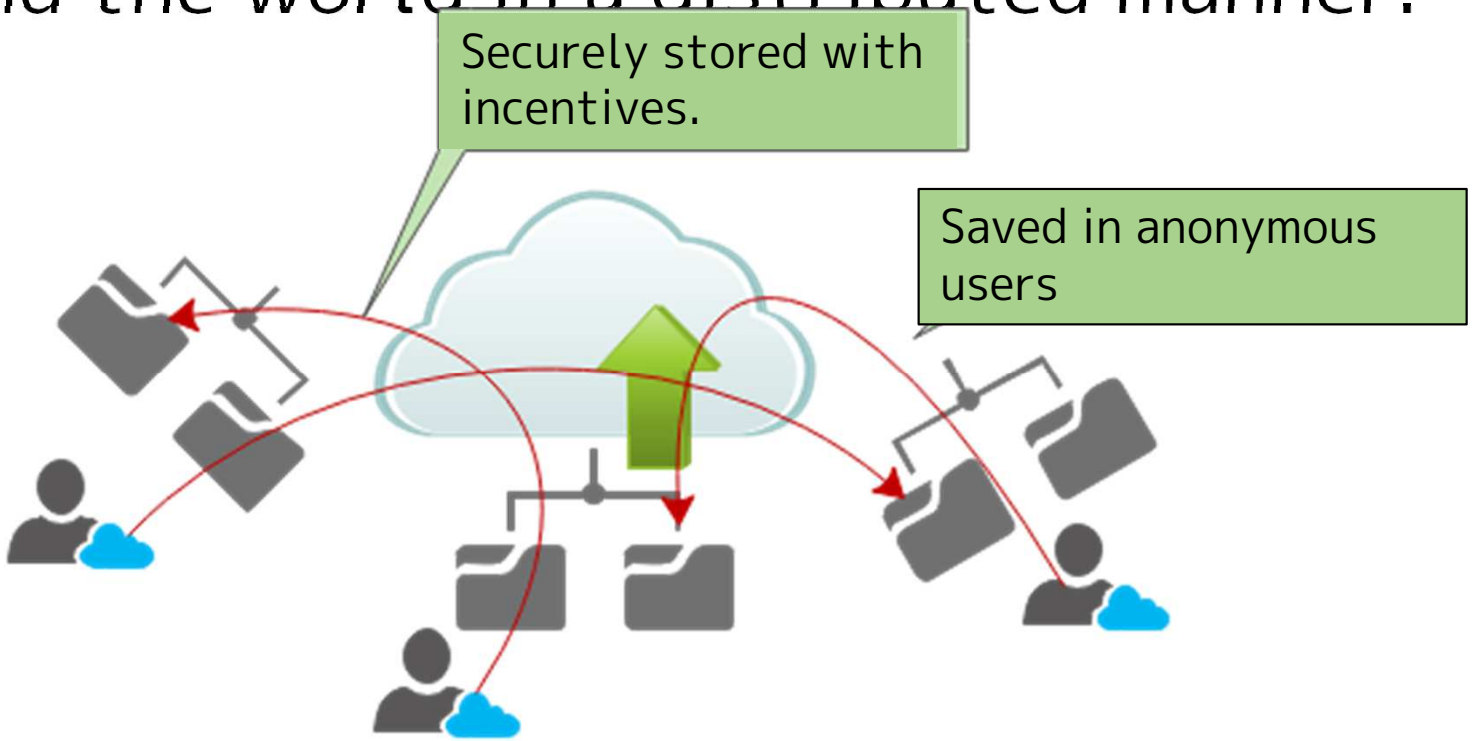
Comparison of SF pre-deployment

- This case is that every VM has containers, i.e., no container pulling is required.
- Even this case, SF-CUV outperforms others.



What is IPFS?

- **IPFS(Interplanetary File System)** is a content-centric P2P middleware for distributed database from Protocol Labs. Inc.
- Every file can be deployed / stored on nodes around the world in a distributed manner.



IPFS commands

- Each peer has its own repository, and then upload a file to other repositories.
  - Register a content in own repository.
    - ipfs add [FILE NAME]

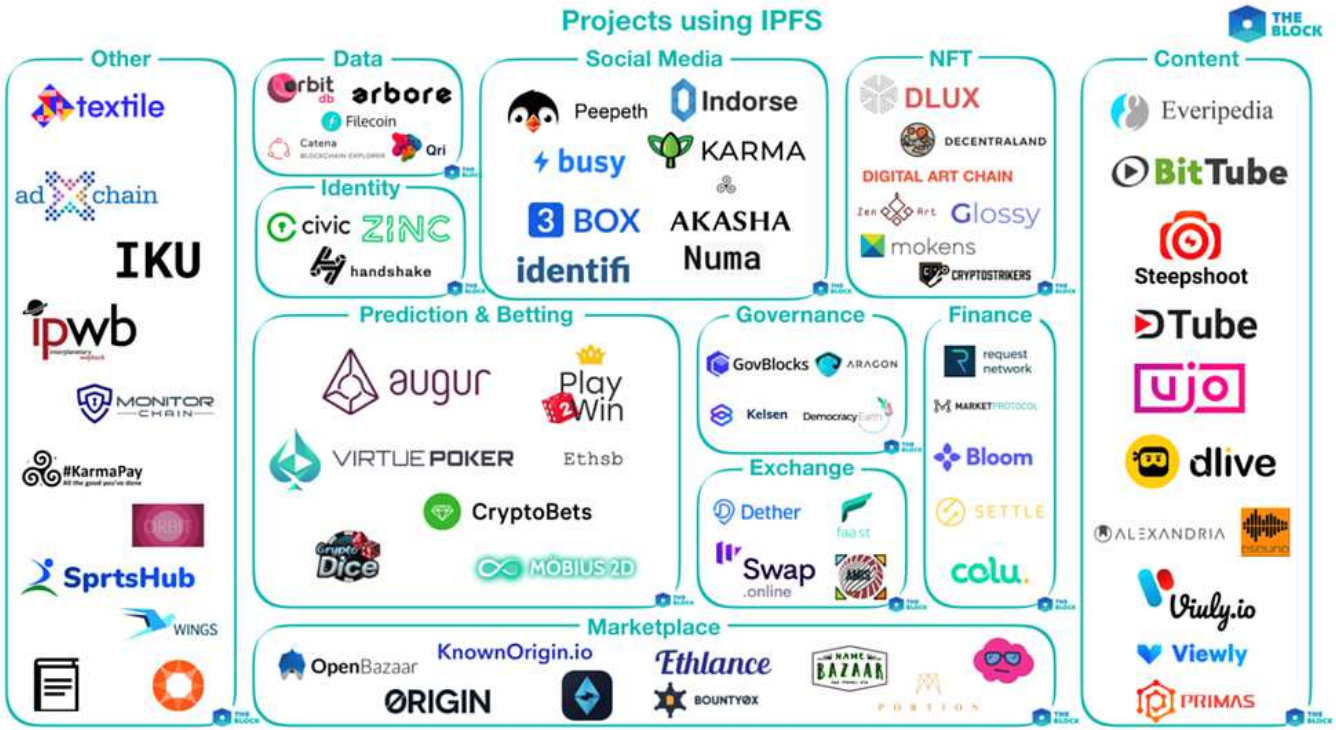
```
[kanemih@ncl-cloud ~]$ ipfs add test.jpg
added QmeuqrKoRRUQPSkD6Qz5EHSgB5xX2Bege2onhEpyVjeWxW test.jpg
[kanemih@ncl-cloud ~]$
```

  - Join IPFS network with own repository is made public.
    - ipfs daemon
  - Display neighbor peers:
    - ipfs swarm peers
  - Get the content provider that holds the content.
    - ipfs dht findprovs [CID]
  - A large con KB chunks and distributed

```
[kanemih@ncl-cloud ~]$ ipfs ls QmeuqrKoRRUQPSkD6Qz5EHSgB5xX2Bege2onhEpyVjeWxW
QmUhLnN64uhXoeSXxdhrauwdK9CX3JAWJedSM7psviBSJG 262144
QmWU8UELmdDm5FVgNLysdc9obTuT9XNbpY5d7zHgPZ77XS 262144
QmTSutEMBAUsez3m9GFxWrfWPdEsoiYbpM3qrkvgkjaj8j 262144
QmSGvJF2G3JUe77q6fS5rSjoRmktSmzZ4sZ6ngGC4esJ8e 262144
QmX2zNrAxpFufkgxCJxgfXncNi24muwDUgheyGAjTbsLzA 262144
QmWH3FrbtbWyTF8LZGzEPaeQK7jMRT8j1mBNrnt2uz3Aw8 262144
QmTtYNdtzUcHrBfKmHHajHxaaKVkv3atRxtgA4sRYivVGM 262144
QmWJd2zHEbvyUd9fX9fdCznwnLydir2dybQ7crk1QieTXU 262144
QmNWBLQbmDUQN5q3DgvXG8M5aEdBrTi81tZRM4mZxSKf7P 262144
QmNVsfVLnxBh39R3nsxZ9TbKvygNzRrmDSZUpQWRRU1BwQ 136854
```

Advantages of IPFS

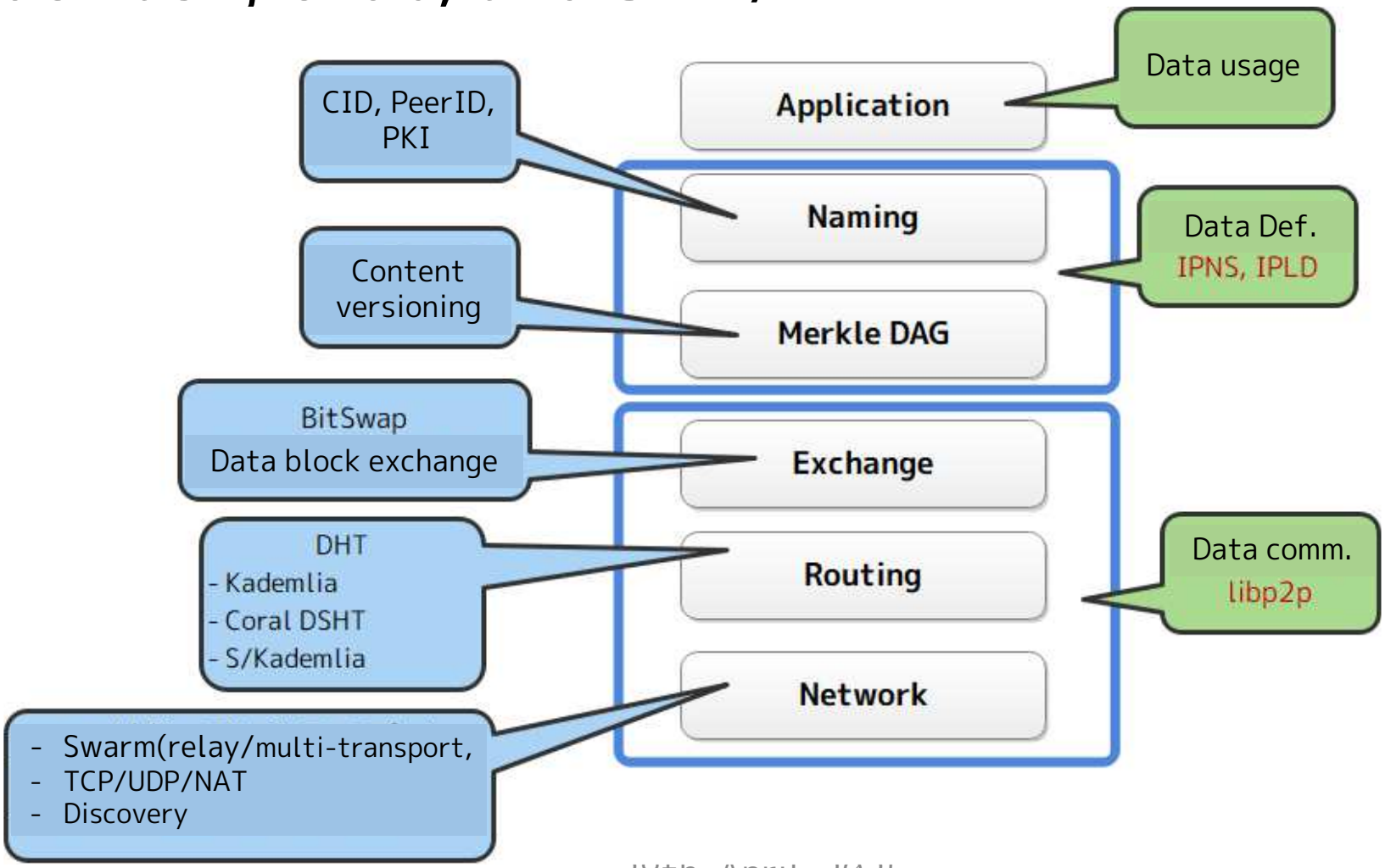
- Each file has CID (Content ID) expressed as a **hash value.**
  - Hash value is derived from the content, not from the name.
- IPFS applications:
  - Ethe
  - Netf
  - Brave



THE BLOCK

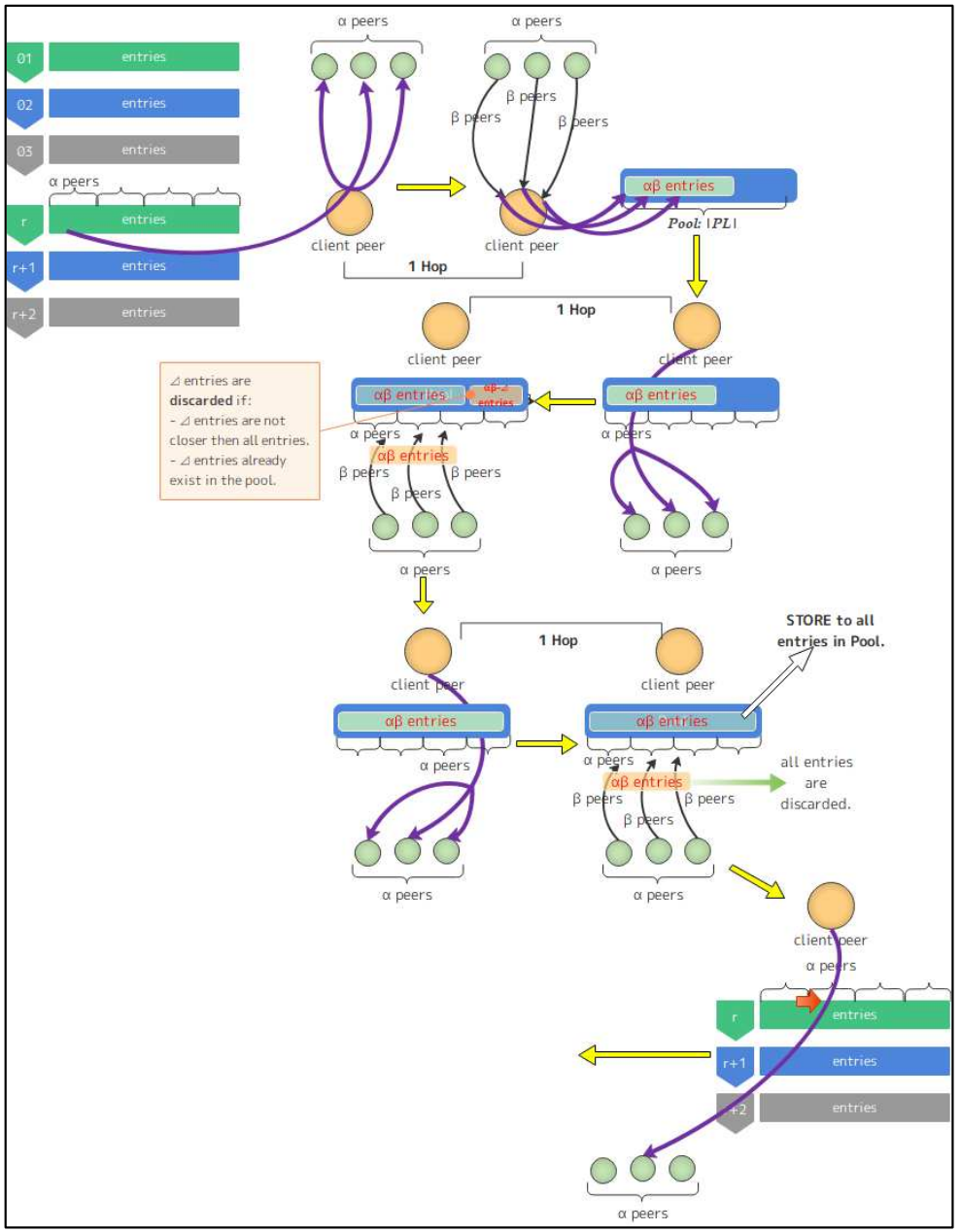
Functional Stack in IPFS

- It consists of IPNS, IPLD (Naming), libp2p (Networking), Parallel downloading, versioning such as github, and DHT/



Content lookup in IPFS (Kademlia)

- Request target are selected in the routing table (k-bucket).
  - α peers are selected.
- A client send lookup requests to α peers in order to make the ID distance (CID/PID) shorter during hops.
  - Distance = XOR among IDs.
- At most **log(Initial Distance)** hops are required to find the provider.
- When α peers are checked and still no content hit, the process goes to the next iteration (queried to the next α peers) .



Research motivation

- IPFS enables us to share various contents transparently among peers.
  - Content routing is based on Kademlia DHT in libp2p.
- Kademlia DHT is known as one of efficient content lookup strategy.
  - Achieves smaller lookup hop count than other DHTs.
  - Simple algorithm and easy to implement.
- However, Kademlia does not aware of:
  - Physical hop count
  - Actual network delay minimization.
- Thus, we proposed KadRTT as a Kademlia alternative.

KadRTT abstract

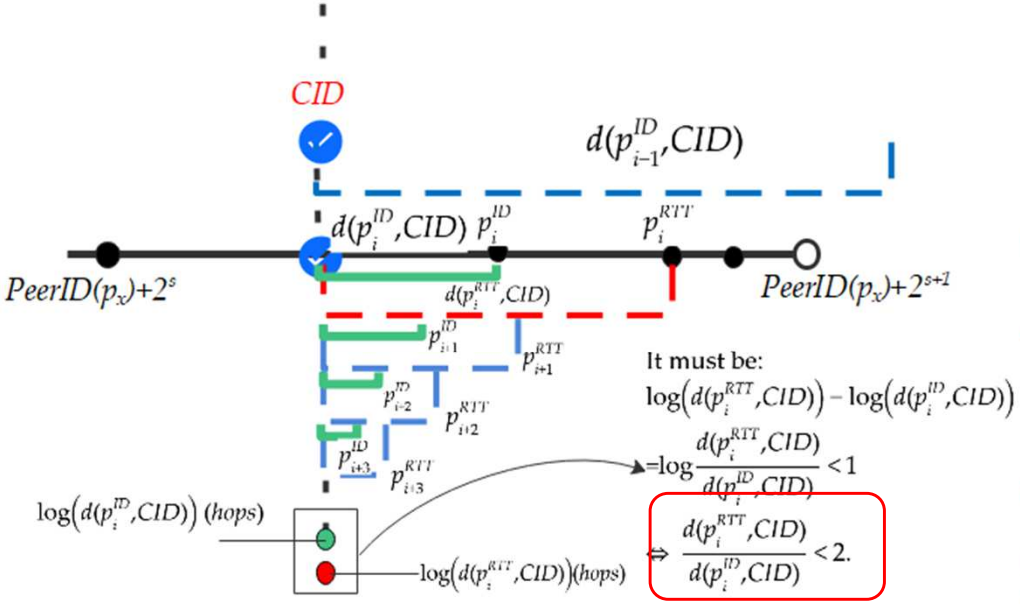
- Objective of KadRTT is to minimize the lookup latency.

- **KadRTT[1]** has two approaches:
  - Delay per hop minimizaiton with guaranteeing total hop bound.
  - Actual total hop count minimization with ID re-arrangement.

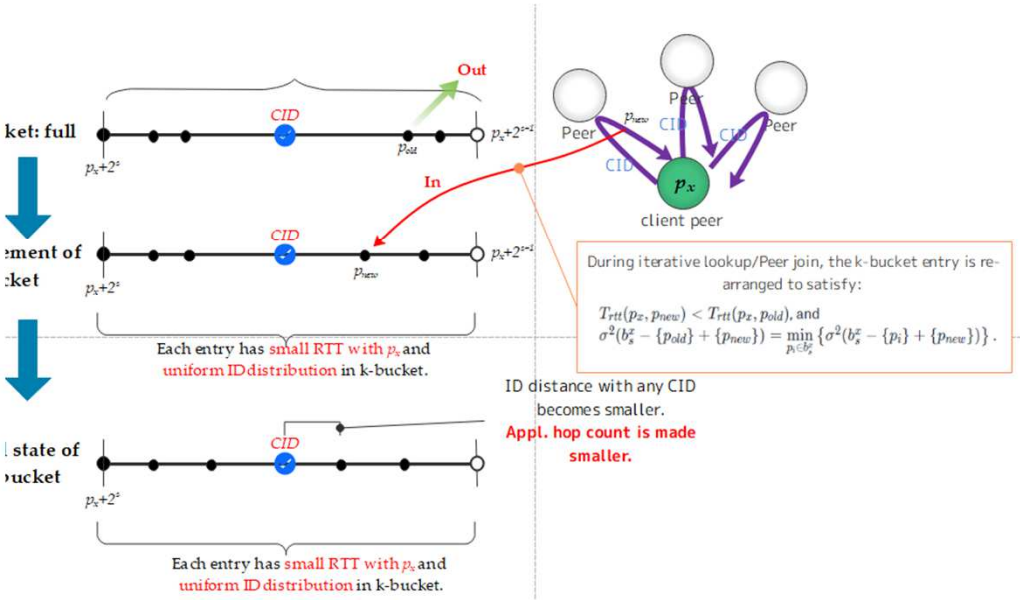
[1] H. Kanemitsu, H. Nakazato, “KadRTT: Routing with network proximity and uniform ID arrangement in Kademlia,” in Proc. IFIP Networking 2021, 2021.

KadRTT: Lookup optimization

- RTT-based lookup and uniform ID arrangement in Kademlia DHT.



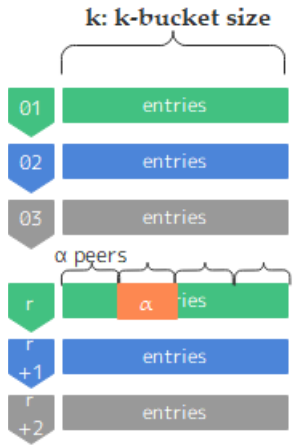
Condition for
hop# of KadRTT= hop # of Kademlia
with minimizing the physical delay.

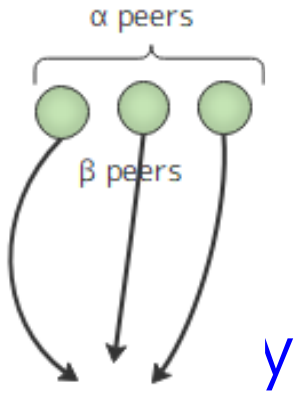


Uniform ID arrangement in k-bucket
to make the initial ID distance
shorter.

KadRTT2: Optimizing parameters in KadRTT

- Automatic adjustment for:
  - k: k-bucket size (# of entries for each index).
    - Affects the initial ID distance (CID / PID).
    - Total hop count:
  - α: Lookup concurrency
    - Affects # of lookup iterations.
    - If α is too large, the network may be congested.
  - β: # of next hops as next query targets
    - Similar to the case of α.

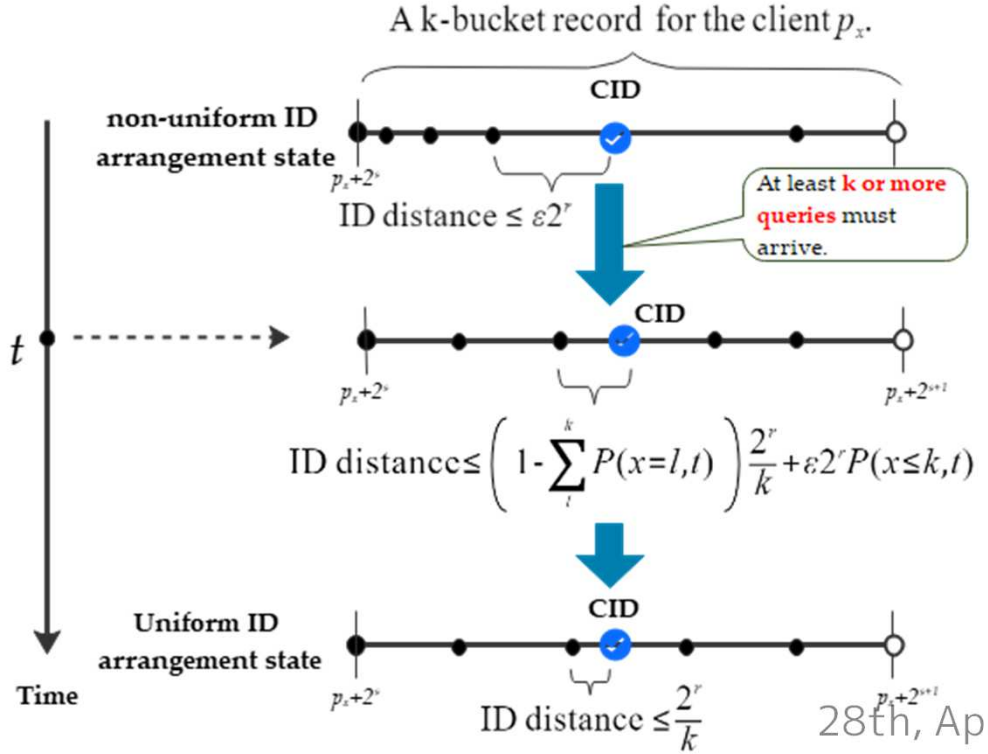




- Why is automatic derivation necessary?
  - Existing routing (e.g., IPFS) set those values y and by rule of thumb. (IPFS sets k=20, α = β=10 as static values).
  - Not optimal.

Derivation for k-bucket size: k

- The id range in i-th k-bucket is $[2^{i-1}, 2^{i})$, and the average distance among peers: $2^{i} / k$.
- This state requires several peer exchage (new/existing one).
- The expected value for the ID distance at given time t is derived.
- Then k is derived when the lookup latency $T_{kadRTT}^{t\to\infty}$ at $t \to \infty$.



$$T_{kadRTT}^{t\to\infty}(r,1) = \rho t_{ave}^{kad} \log\left(\frac{\lambda^k e^{-\lambda} 2^r}{k}\right).$$

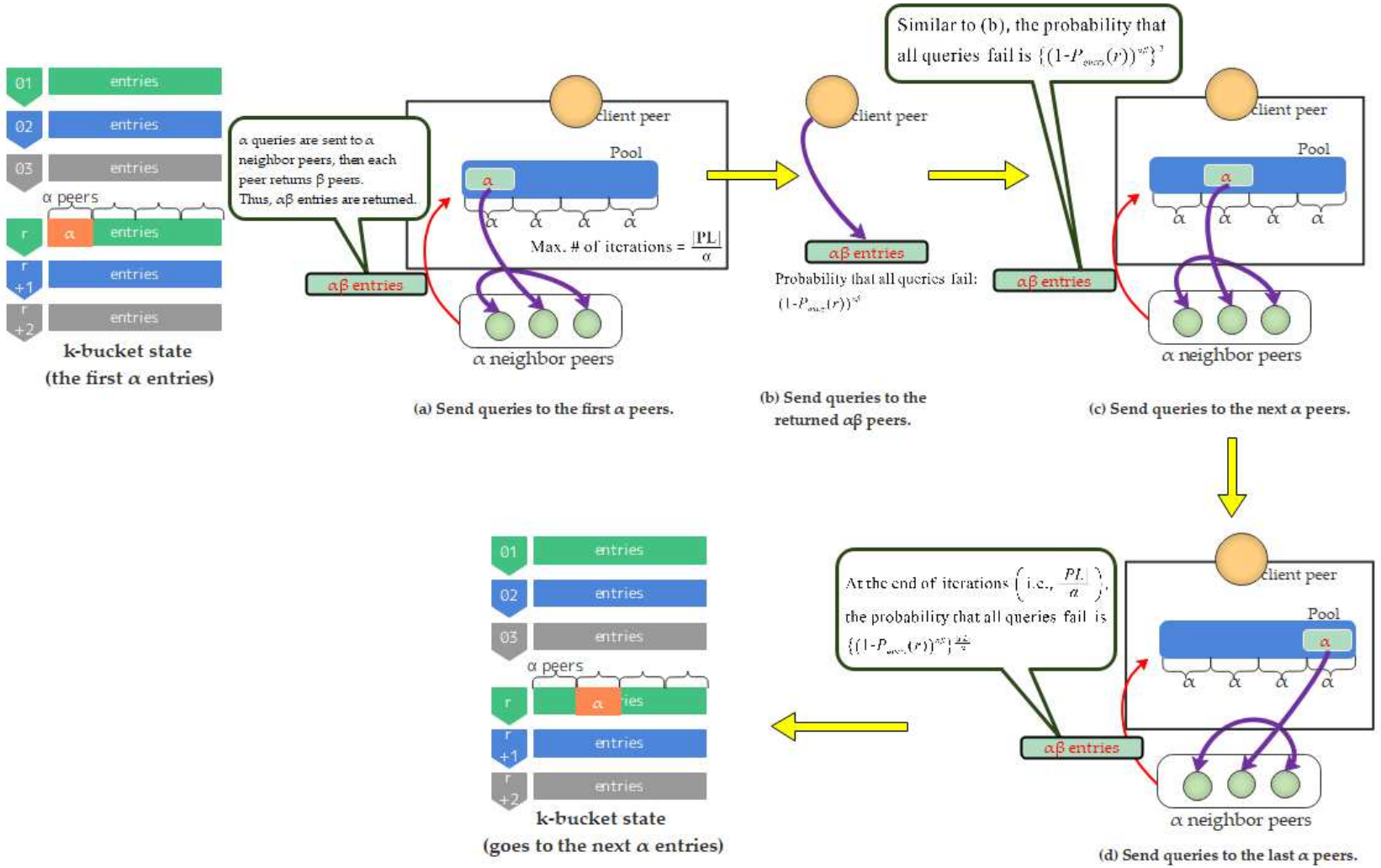
$$k_{opt}(r) = \left\lceil -\frac{W(-2^r e^{-\lambda}\log_e \lambda)}{\log_e \lambda} \right\rceil.$$

where W: Lamber W function s.t.,

$$f(x) = xe^x \text{ and } f^{-1}(x) = W(x)$$

Example of lookup failure.

- In k-bucket, when no content provider found from α neighbors, the queries goes to the next α peers.
- "Iteration" is # of transactions in the k-bucket.



Derivation for α / β

- Expected value for # of iterations is defined as $\mathrm{E}_{\mathrm{ite}}(\alpha, \beta, k)$.

$$E_{ite}(\alpha, \beta, k) = 1 - \frac{k_{opt}(r)}{\alpha} \left(1 - P_{query}(r)\right)^{\frac{k_{opt}(r)}{\alpha}\beta|PL|},$$

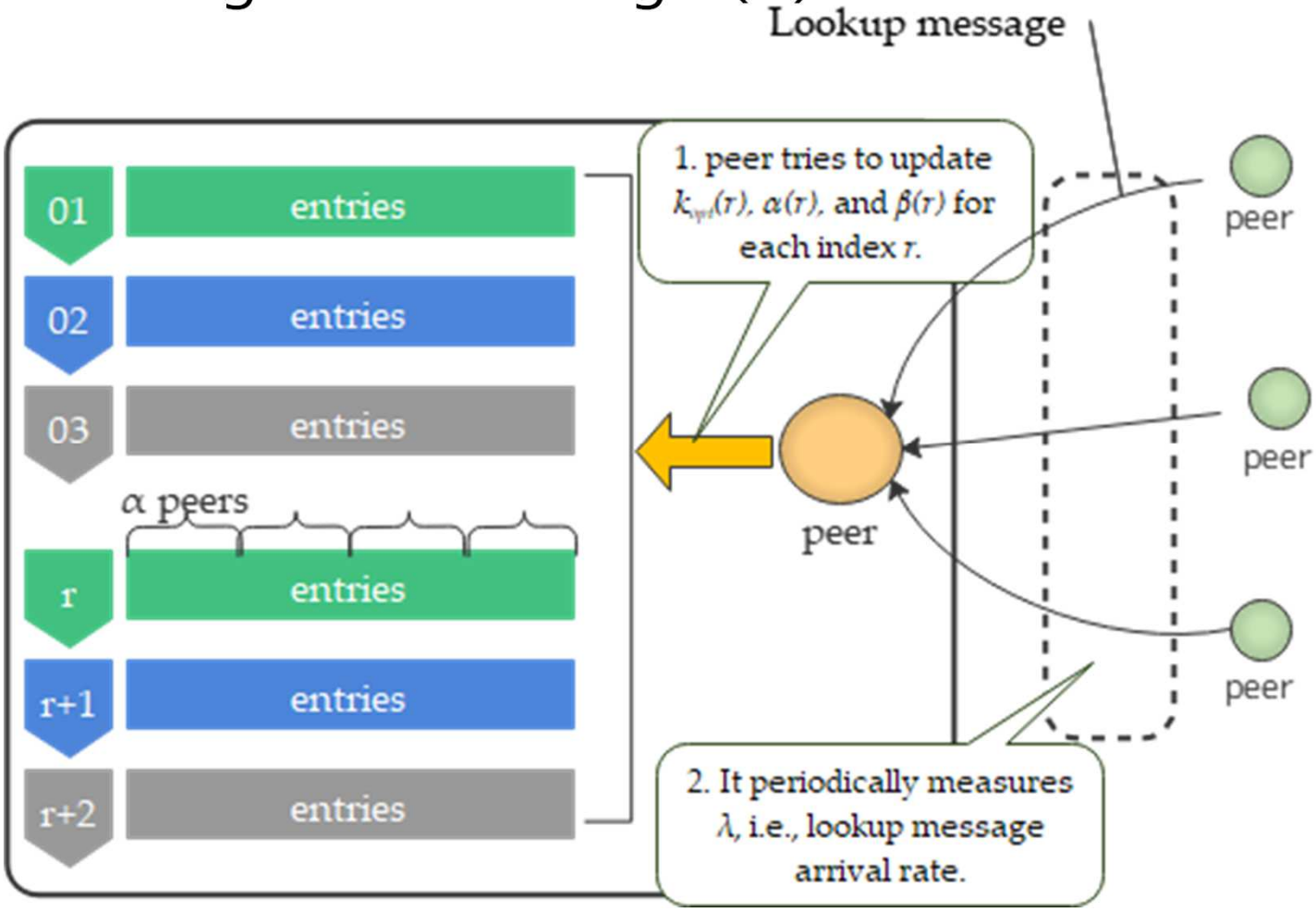
- Both α and β are derived when $\mathrm{E}_{\mathrm{ite}}(\alpha, \beta, k)$is minimized.

$$\alpha(r) = \left\lceil |PL| \sqrt{\left| \frac{2k_{opt}(r) \log_e \left(1 - P_{query}(r)\right)}{W\left(\frac{2}{k_{opt}(r)} |PL|^2 \log_e \left(1 - P_{query}(r)\right)\right)} \right|} \right\rceil,$$

$$\beta(r) = \left\lceil \sqrt{\left| \frac{W\left(\frac{2}{k_{opt}(r)} |PL|^2 \log_e \left(1 - P_{query}(r)\right)\right)}{2k_{opt}(r) \log_e \left(1 - P_{query}(r)\right)} \right|} \right\rceil.$$

Actual parameter adjustment

- Each peer periodically update k/α/β to reflect the latest state to each query.
- Also, message arrival rage (λ) is also measured.



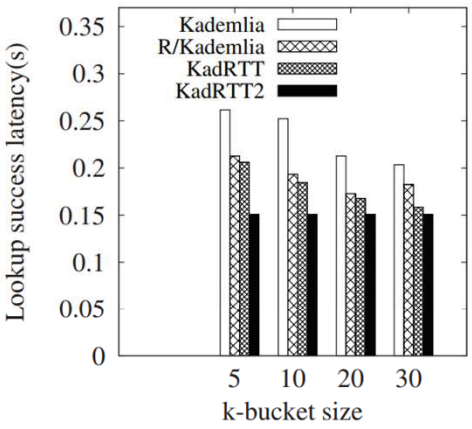
Experiment by simulation

- Firstly, we implement KadRTT2 in OverSim, which is plugin of overlay communications for omnet simulator.
- Comparison target: Kademlia, R/Kademlia, KadRTT

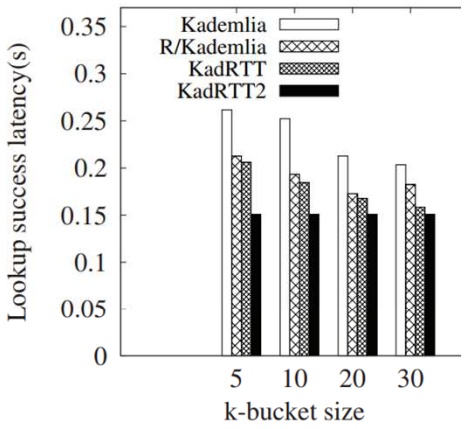
Parameter	Value
♯ of peers	3000
Sim. time (s)	500
Churn model	Pareto churn [9]
Lookup concurrency (α)	variable
♯ of next hops (β)	variable
k-bucket size (k)	variable
Lookup type	Iterative lookup
Measurement interval for KadRTT2 (s)	50
Content hit probability ($P_{query}(r)$)	0.98
Application type	KBRTest

Lookup latency

- In (k, α, β), any one is fixed / others are variable.
- In any cases, KadRTT2 outperforms others in terms of the lookup latency.

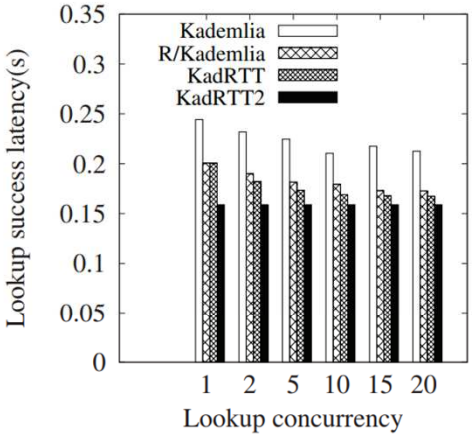


(a) $(\alpha, \beta) = (5, 10)$.

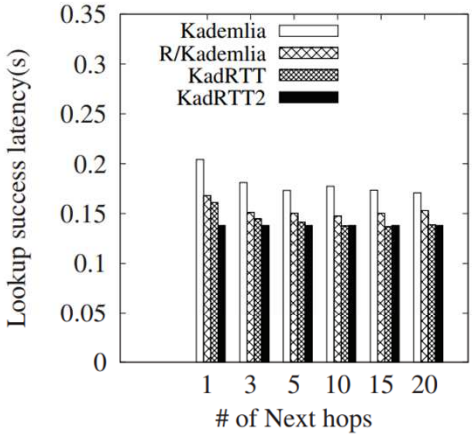


(b) $(\alpha, \beta) = (10, 5)$.

Fig. 6. Lookup Latency with Varying k.



(a) Varying α with $k = 20$.

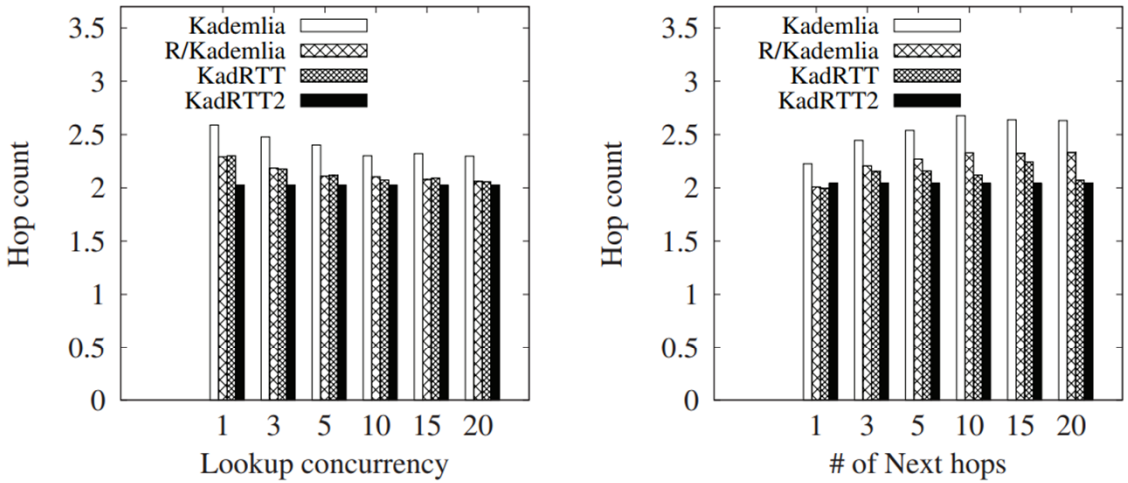


(b) Varying β with $k = 20$.

Fig. 7. Lookup Latency with Varying α and β.

Hop count

- As for the hop count, it is slightly improved from KadRTT.
- We can say that the derived value for k is appropriate.
- Though smaller # of hops cannot always lead to lookup latency minimization, the initial ID distance (CID, PID) is made smaller than that of other approaches.



(a) Varying α with $k = 20$. (b) Varying β with $k = 20$.

Fig. 8. Hop Count with Varying α and β.

Emulation on many containers.

- We conducted emulation by “testground”.
- Each peer is a docker container, and managed by EKS (Aazon Elstic Kubernetes Service).
- KadRTT/KadRTT2 is implemented on libp2p (actual code), and compile in each containers.

Parameter	Value
♯ of peers (containers)	300, 600, 1200
runner	cluster:k8s
Test case	“find-providers”
Network latency (ms)	100
k-bucket size (k) for kad-dht/KadRTT	20
Lookup concurrency (α) for kad-dht/KadRTT	10
♯ of next hops (β) for kad-dht/KadRTT	10
Content hit probability for KadRTT2 ($P_{query}(r)$)	0.98
Measurement interval for KadRTT2(s)	50

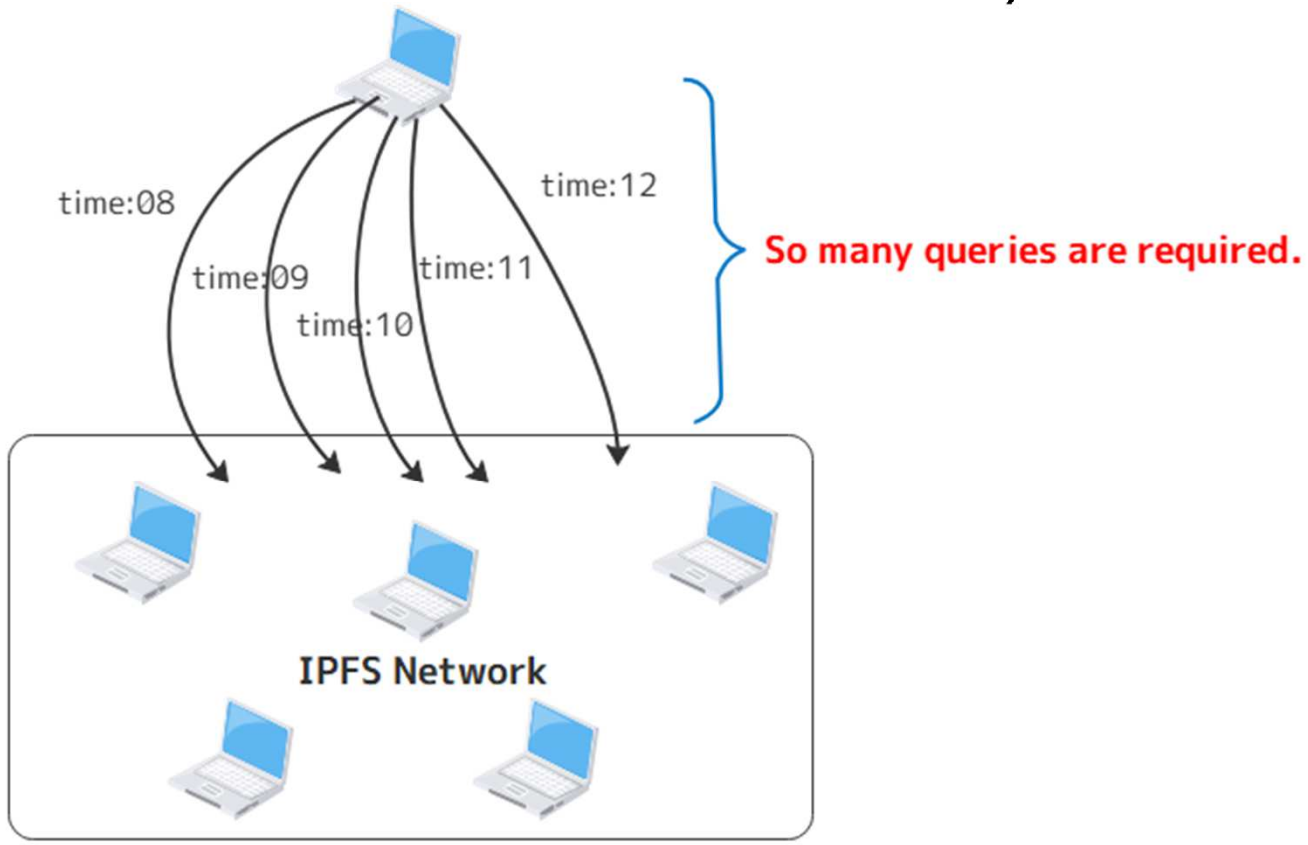
Evaluation results

- Though only limited number of patterns are tested, the latency in KadRTT2 outperforms KadRTT.
- However, # of queries in KadRTT2 is higher than that of KadRTT.
  - Each parameter (α, β, k) may take higher value than default ones, thereby many communications among client / neighbors are required.

	FindFirst(avg.)			Rit Rate			♯ of queries		
♯ of peers	300	600	1200	300	600	1200	300	600	1200
kad-dht	2824.361	3287.5562	4411.6519	73.037	85.341	82.123	28288	39943	52163
KadRTT	2345.854	2594.5145	3422.601	99.684	99.443	99.567	20345	28239	36845
KadRTT2	1819.162	1993.8016	2512.2627	99.493	99.667	99.384	21146	29055	39141

Complete match search in IPFS

- In complete match search (CMS), DHT assumes the required information is previously determined by the user.
- If multi-attribute / range query is done by DHT, many queries must be sent and it leads to heavily loaded.



Case: Contents with "time:08 - 12" are traced by DHT.

Why DHT cannot support flexible queries?

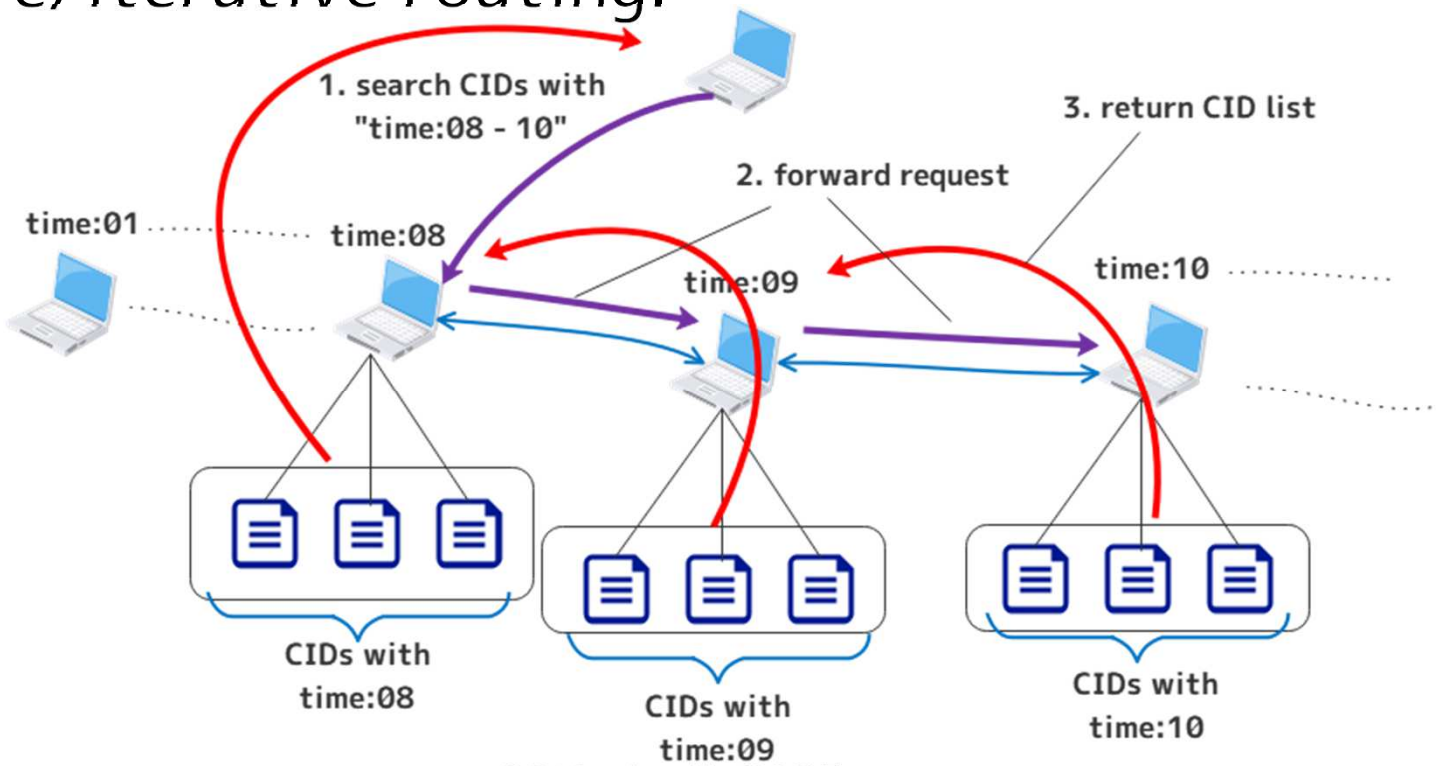
- **No relationship between the hashed ID and actual values.**
- All nodes must be queried if all contents for the multi-attribute/range search conditions.
- Current approaches for the flexible queries in P2P:
  - **Mercury**
    - Each raw data is deployed to nodes, if bias exists among data, some of them are moved to others.
    - The deployment target nodes are determined by the degree of congestion in neighbor nodes.
  - **PHT (Prefix Hash Tree)**
    - Each attribute are distributed among nodes by hashing each node's prefix.

Why flexible query?

- Current complex network such as IoT, variety of user's requests makes DHT handle multi-attribute queries.
- **IoT network：**
  - A specific sensed data in the specific time duration is required.
  - To acquire data at the specific location and process statistics for it.
- **LiDAR application using the network：**
  - To acquire the point cloud map at the specific location.

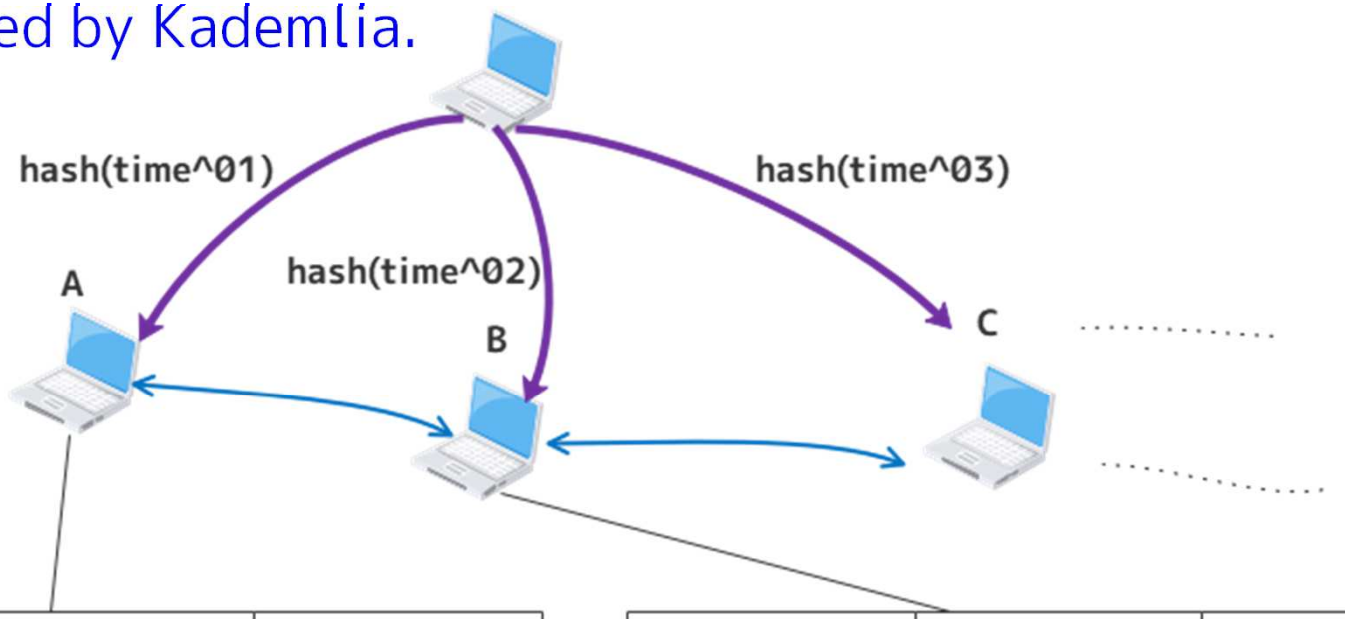
Basic idea in the proposal

- Add the “new nodes” corresponding to attributes to “Merkle DAG” in IPFS.
  - As a result, each content has one or more attributes.
- (attr_name, attr_value) is put to management nodes, and bi-directional links are established among them.
- Recursive/iterative routing.



Deployment of attributes

- Attributes should be deployed on management nodes in advance.
  - Ex., time:01~time:24 for each management node.
- Management nodes are determined by calculating Hash(time^01)~ Hash(time:24) with the shortest XOR distance with them.
  - Determined by Kademlia.

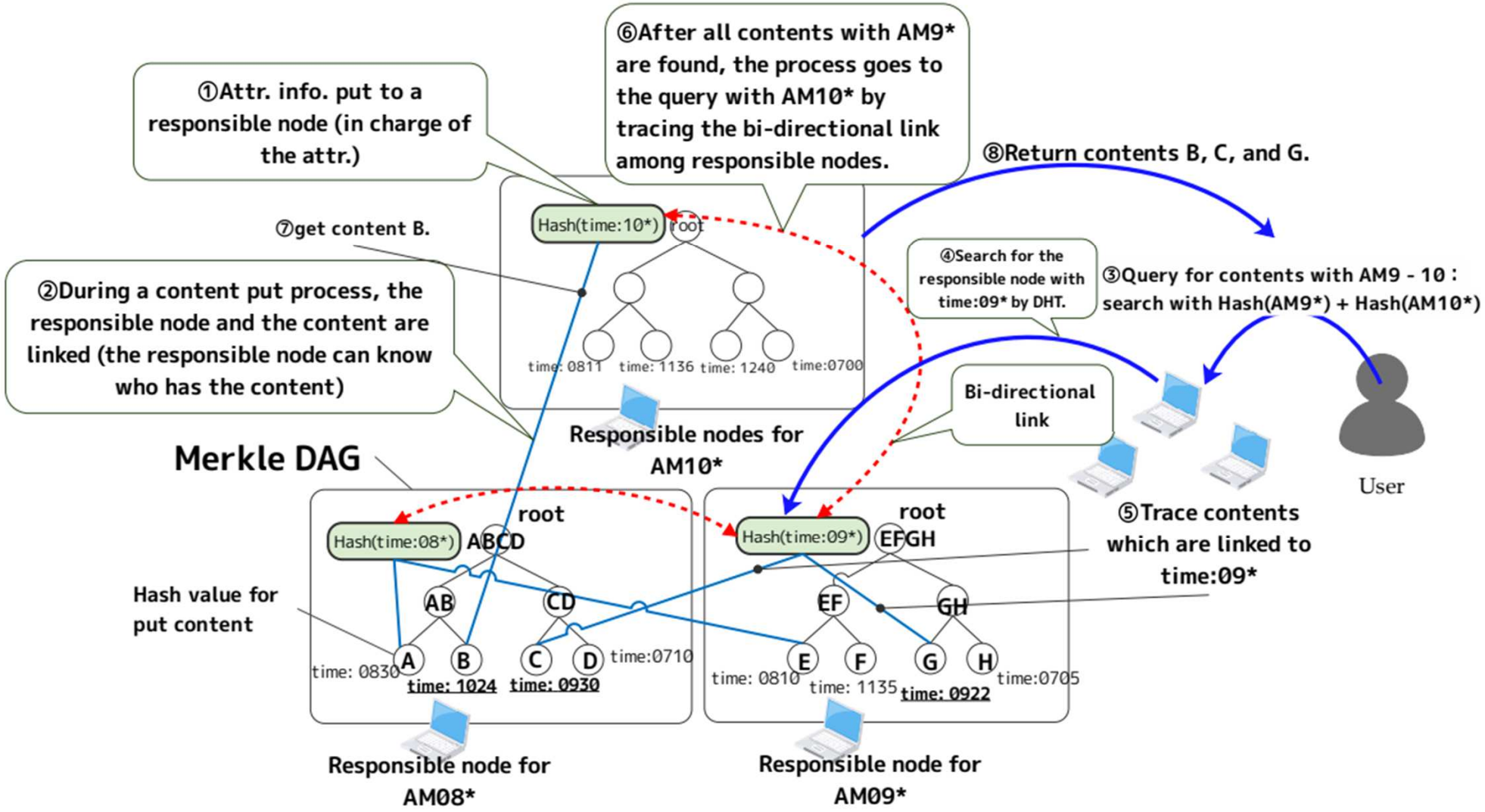


Attr	Predecessor	Successor
hash(time^01)	null	B

Attr	Predecessor	Successor
hash(time^02)	A	C

Overall procedures

- By adding CIDs to Merkle DAG for each management node, any content can be traced with specific attributes.



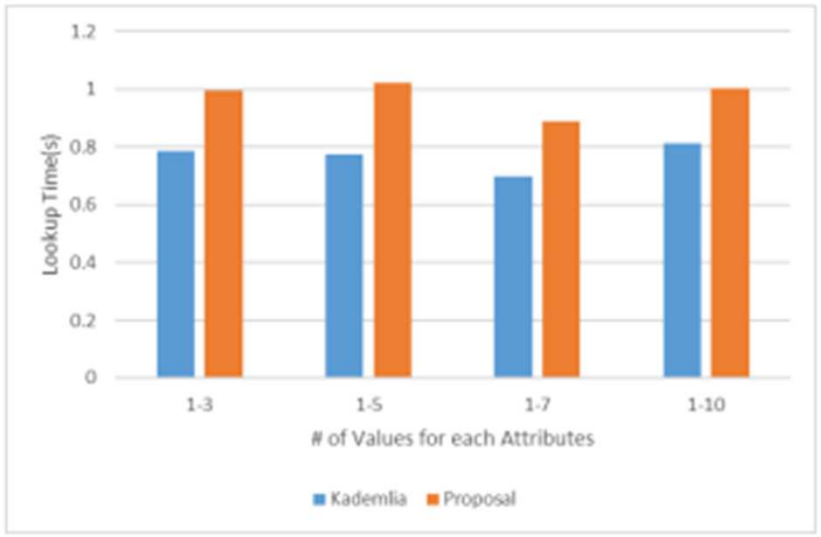
Experiments

- We conducted evaluation in terms of:
  - Single query performance
  - Lookup performance with varying the value range.
  - Lookup performance with varying the number of attributes.
- The experiments were conducted in the real-world network as following parameters

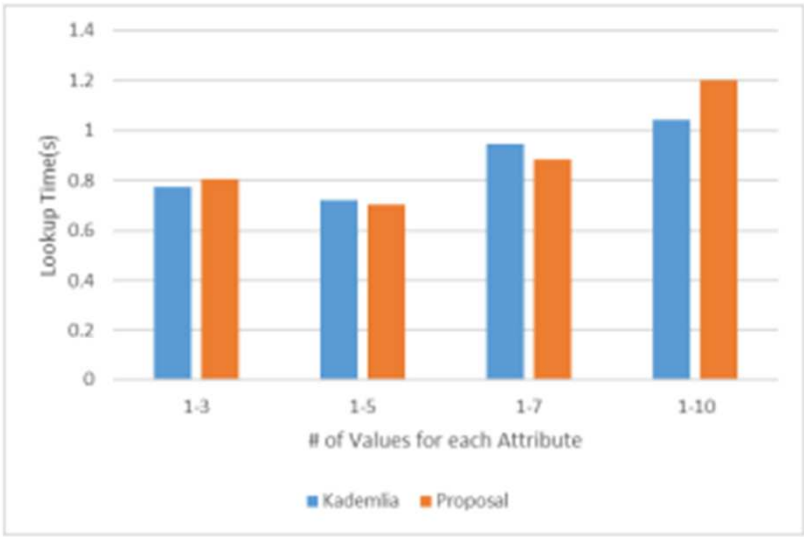
Parameter	Value
♯ of nodes	33
VM instance	t2.large (2vCPU, 8GB RAM)
♯ of Attrs.	1 – 5
Range of values per attr.	1 – 10
Emuration time	20 min
Prob. for requests per minite	0.1(10%) - 0.8(80%)

Single query performance

- The performance as the lookup time in the proposal is worse than Kademlia because of the existence of management nodes.



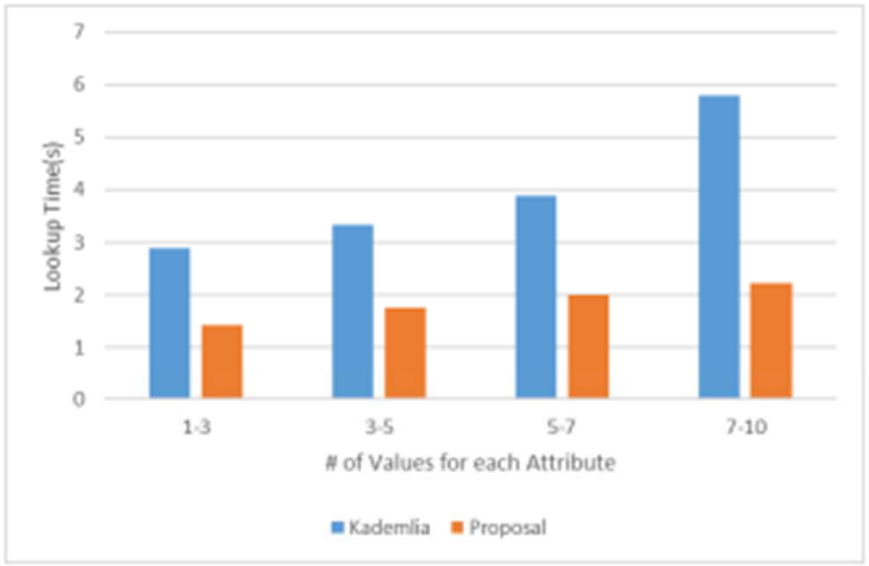
(a) ♯ of attrs. 1–3.



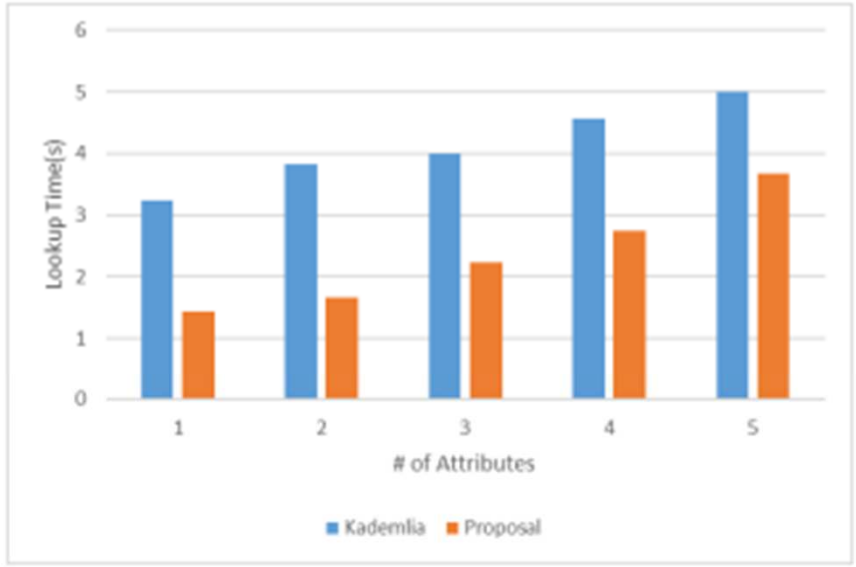
(b) ♯ of attrs. 1–5.

Multi-Attribute and Range Query Comparisons

- In (a) and (b), our proposal outperforms Kademlia DHT.
- In (a), the difference in terms of the lookup time among two algorithms is made larger if the number of values for each attribute increases.



(a) ♯ of values per attr.

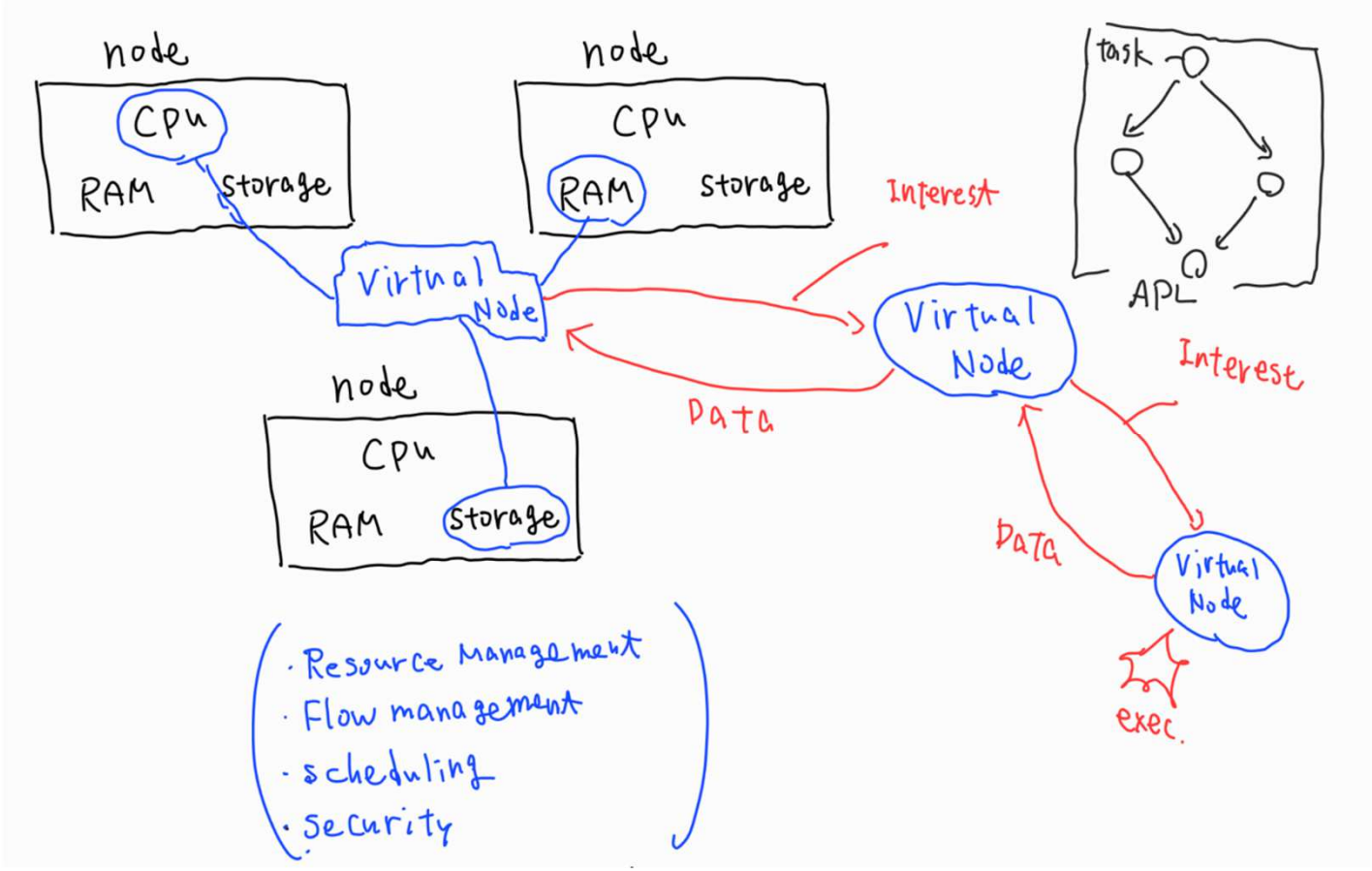


(b) ♯ of attrs.

Currently proposing project

Disaggregated computing with decentralized resource management

- Each element is virtually shared among nodes to generate a “virtual node”.
- Each virtual node collaboratively executes any kind of application.



Discussion

- What is truly-distributed computing?
  - To make CPU/RAM/storage transparent to users.
  - A distributed OS is required.
- No management node is required.
- How to realize a secure system?
  - Blockchain is required.

Thank you!!

Hidehiro Kanemitsu,
Associate Professor,
Global Education Center, Waseda University,
Japan
kanemih@waseda.jp